

Time Series Data

Definition

Time series data refers to data where the same variable is observed repeatedly over different points in time. The observations are arranged in chronological order, and the time component plays a very important role. What happened in the past can influence what happens in the present and the future. Because of this, the order of the data cannot be changed without losing meaning.

The time interval can vary depending on the application. It may be seconds, minutes, hours, days, months, years, decades, or even centuries. A key characteristic of time series data is dependency over time. Consecutive observations are often related to each other, which makes time series analysis different from other types of analysis.

Examples

One example is recording the closing stock price every day for 100 days. Another example is tracking stock price changes every 15 minutes during a single trading day. We could also record the average temperature every day or measure temperature every hour.

In all these cases, the value at a given time is usually influenced by previous values. For instance, today's stock price is generally related to yesterday's price. Even though there may be random fluctuations, there is typically a noticeable dependency on past observations.

Cross-Sectional Data

Definition

Cross-sectional data refers to data collected from many different units at a single point in time. In this case, time does not play a structural role in the dataset. Instead of studying changes over time, we compare differences across individuals, objects, or entities. The observations are not ordered in time, and there is usually no dependency between them based on sequence.

Examples

A simple example is throwing a dice 100 times. Each outcome is independent of the previous one, and there is no time-based relationship between throws.

A more practical example is collecting data on 1,000 houses in Dharwad on a particular day. For each house, we record the sale price, size, number of rooms, location rating, and age of the building. Now the question is can we predict the housing price using the features? In this, we are working with cross-sectional data. All observations are collected at one point in time, and there is no time dimension involved.

Panel Data (Longitudinal Data)

Definition

Panel data, also called longitudinal data, is a combination of time series and cross-sectional data. In panel data, we observe many different units and track each of them over multiple time periods. In other words, we study several subjects repeatedly across time.

Here, both dimensions are important: the differences between units and the changes over time. Because of this structure, panel data allows us to analyze not only how something evolves over time but also how different entities behave differently under similar conditions. It captures time dependency as well as individual-specific characteristics.

Practical Example

Practical example is collecting house price data for 1,000 houses in Dharwad every year for 5 years. Now we can study both how house prices change over time and how prices differ across houses. This combined structure makes the dataset panel data.

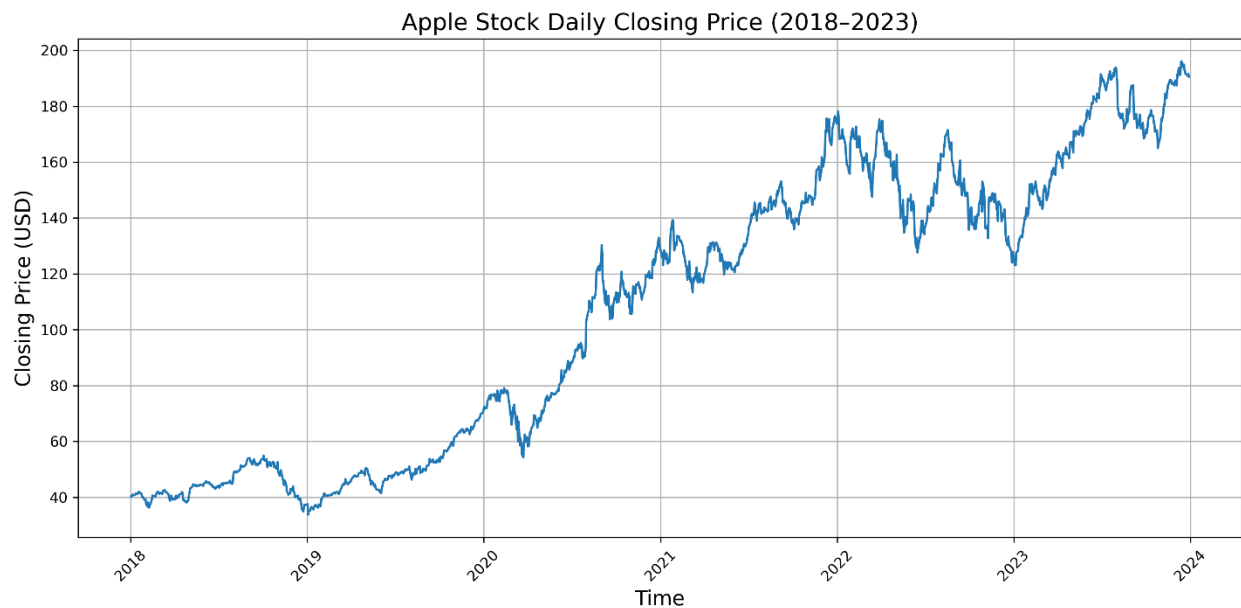
In simple terms, panel data means studying many subjects over many time periods, allowing us to analyze both time dynamics and cross-sectional differences together.

Let's see some examples we want to discuss throughout the class.

1. The Airline Passengers dataset is a monthly time series that records the total number of passengers traveling over several years. Each observation corresponds to one month, making time the central component of the data.



2. The Apple stock price dataset consists of the daily closing prices of Apple Inc. from 2018 to 2023. Each data point represents the stock's value at the end of a trading day, forming a high-frequency financial time series.



What we are going to do with time series data? We want to do forecasting.

Forecasting means estimating what future values of a variable are likely to be, using its past behavior and patterns. In simple terms, we look at historical data, understand how it has behaved, and then use that information to make informed predictions about what may happen next.

In the Airline Passengers data, forecasting means predicting how many passengers are likely to travel in upcoming months. This helps airlines plan flight schedules, staffing, and capacity requirements.

In the daily closing prices of Apple Inc., forecasting means estimating future stock prices or price movements. This can support investment decisions, risk management, and trading strategies. We can make money!

How to forecast? Look at the waveform and observe

Observations for both data

Airline Passengers Data: The number of passengers increases steadily over the years, and the rise becomes more pronounced as time progresses. The fluctuations within each year repeat in a fairly regular pattern, with certain months consistently showing higher passenger counts than others. In the later years, both the overall passenger numbers and the size of the fluctuations become larger, indicating stronger growth and wider variation compared to the earlier period.

Apple Stock Price Data (2018–2023): The closing price of Apple Inc. shows substantial movement over the six-year period, with phases of sharp increases followed by noticeable declines. The price does not move smoothly; instead, it exhibits frequent short-term ups and downs, along with occasional large jumps or drops. Certain periods display relatively calm behavior, while others show intense price swings, reflecting changing market conditions and investor reactions.

Usually we decompose signal into three parts

Trend: The trend shows the overall long-term direction of the data after smoothing out short-term fluctuations.

Seasonality: Seasonality captures repeating patterns that occur at regular intervals.

Residual: The residual is what remains after removing the long-term movement and repeating patterns. It represents irregular or unexpected variations.

Lets extract these from the data (we will discuss how to extract these later)

Airline Passengers Data Observations

Trend: In the airline passenger data, the long-term movement shows a steady rise in the number of passengers over the years. The growth becomes more pronounced in the later period, indicating increasing air travel demand over time.

Seasonality: The data shows a clear repeating yearly pattern. Certain months consistently record higher passenger counts, while others show lower values. This repeating behavior occurs every year in a similar manner.

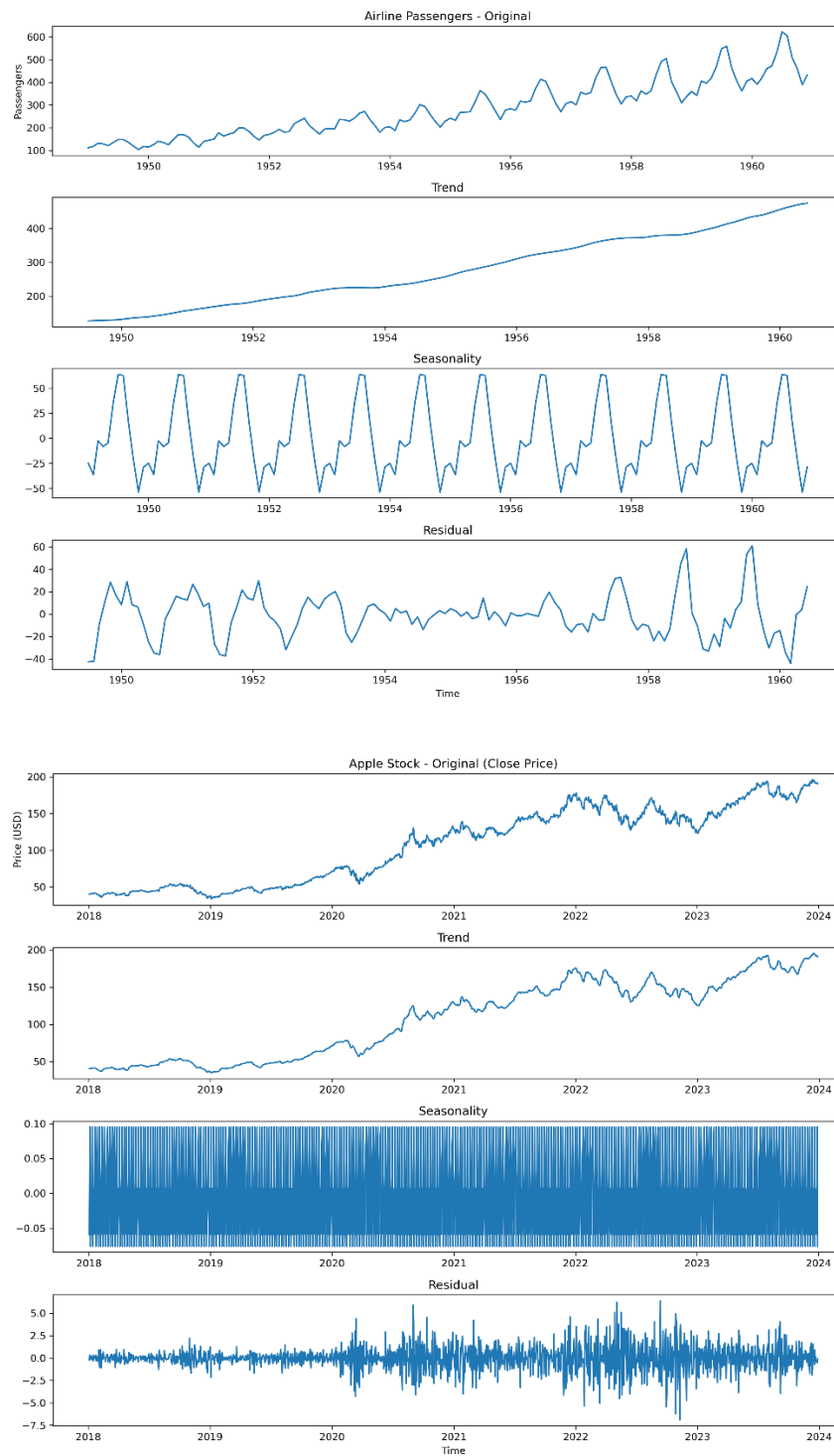
Residual: After removing the long-term increase and the repeating yearly pattern, the remaining variation reflects irregular changes. These may be due to unexpected events, economic factors, or random fluctuations that are not systematically repeated.

Apple Stock Price Data (2018–2023)

Trend: For Apple Inc., the long-term movement in closing prices shows extended periods of growth along with phases of decline. This component reflects the broader valuation movement of the company over several years.

Seasonality: In daily stock prices, any repeating short-term structure is much weaker compared to airline data. Small recurring effects may appear within trading cycles, but they are less consistent and less pronounced.

Residual: Once the long-term movement and any repeating short-term structure are removed, the remaining part represents sudden price changes and daily market noise. This includes reactions to earnings reports, macroeconomic events, or unexpected news.



Which one is difficult to predict and why?

The Apple stock price data is generally more difficult to forecast than the airline passenger data because it is influenced by many unpredictable external factors such as market sentiment, global

economic events, company announcements, and sudden news. While airline passenger data follows relatively stable and repeating patterns over time, stock prices exhibit frequent sharp movements and higher irregular fluctuations. This makes financial time series more volatile and less structurally stable, increasing uncertainty in predictions and making accurate forecasting more challenging.

Naive forecasting methods or Bench marking forecasting methods

Naive methods are very simple forecasting techniques. They are often used as starting points to compare with more advanced models. If a complex model cannot perform better than these simple methods, then it may not be useful.

1. Mean Method:

This method predicts all future values as the average of the past data. It assumes the data stays around a constant level without major changes.

2. Naïve Method:

This method says the next value will be the same as the most recent value. In simple terms, “tomorrow will be like today.”

3. Seasonal Naïve Method:

This method uses the value from the same period in the previous cycle. For example, the forecast for next January is the value from last January.

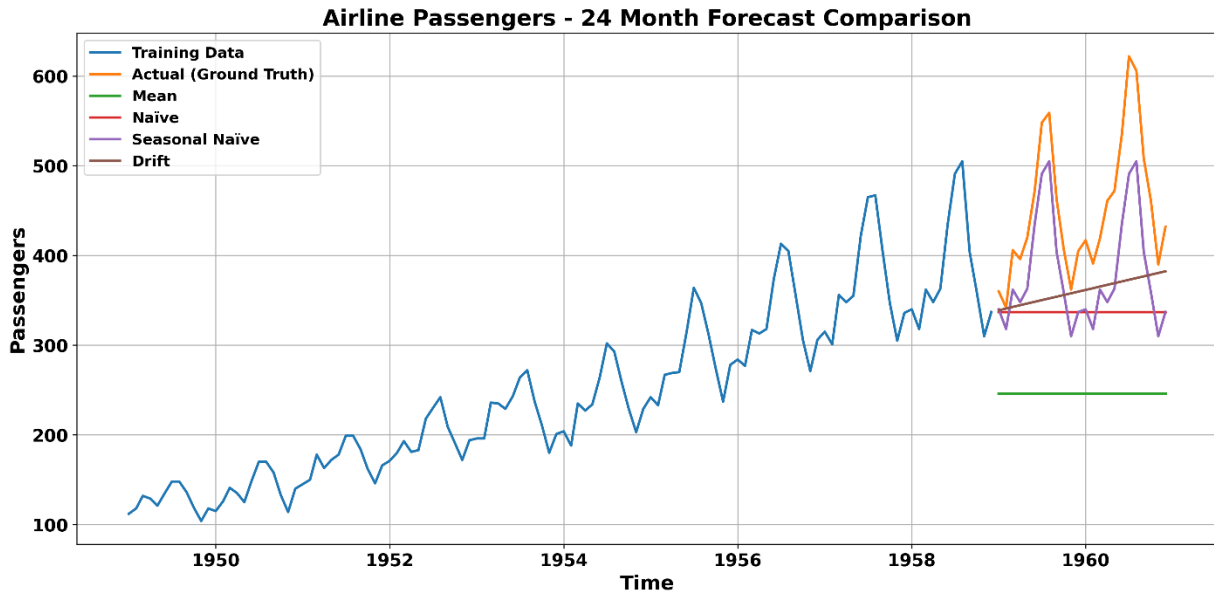
4. Drift Method:

This method assumes the data has been increasing or decreasing at a steady rate. It draws a straight line from the beginning to the latest point and extends that line into the future.

Lets see the results for these methods below.

Airline Passengers Data: For the airline data, the seasonal naïve method performed the best among the four approaches, as it was able to closely follow the actual passenger values. The mean method performed the worst because it ignored the structure in the data and simply predicted a constant value. The simple naïve method did better than the mean method, and the drift method performed slightly better than the naïve method. However, all the methods generally underestimated the actual passenger numbers during the forecasting period.

Apple Stock Data: For the Apple stock data, all the methods performed poorly compared to the actual values. The mean method was again the worst because it failed to capture changes in price levels. The naïve and seasonal naïve methods performed almost similarly, with neither clearly outperforming the other. The drift method tended to overestimate the prices, while the mean method underestimated them. Overall, forecasting the stock data was much more difficult than forecasting the airline data.



Forecast Horizon

The forecast horizon refers to how far into the future we want to predict. For example, predicting the next month is a 1-step-ahead forecast, while predicting the next 12 months is a 12-step-ahead forecast. As the horizon increases, uncertainty also increases because we are moving further away from the observed data. In practical terms, short horizons are usually more accurate, while long horizons are more challenging because small errors can accumulate over time.

Multi-Step Forecasting Using Recursive Estimation

In recursive forecasting, we build a model to predict only one step ahead. Then we use that prediction as an input to forecast the next step, and continue this process repeatedly until we reach the desired horizon. For example, to predict 12 months ahead, we first predict month 1. Then we use the predicted value of month 1 to predict month 2, and so on. This method is simple and widely used, especially in classical time series models. However, errors can accumulate at each step, which may reduce accuracy for longer horizons.

Multi-Output (Direct) Forecasting Method

In the multi-output method, commonly used in machine learning, the model predicts multiple future steps at once. Instead of predicting one step and feeding it back, the model directly outputs a vector of future values, such as the next 12 months simultaneously. This approach avoids error accumulation because each future step is predicted independently of previous forecasts. It is especially popular in neural networks and other machine learning models. However, it may require more data and careful training to perform well.

Moving Average Method

The moving average method improves on the simple average by using only the most recent k observations instead of the full history. This allows the forecast to adapt more quickly to recent changes in the data. The moving average uses only the most recent observations, making it more responsive to changes in the data. Because of this, it is generally more practical for forecasting time series where recent values are more informative than older ones.

Mathematically, if we have observations $y_1, y_2, \dots, y_T$ the forecast for the next period is:

Simple average:

$$\hat{y}_{T+1} = \frac{1}{T} \sum_{t=1}^T y_t$$

Moving Average:

$$\hat{y}_{T+1} = \frac{1}{k} \sum_{i=0}^{k-1} y_{T-i}$$

Exponential Average Method

The exponential moving average is an improvement over the simple moving average because it gives more weight to recent observations and gradually reduces the weight for older observations. This makes the forecast more responsive to recent changes while still using

information from the entire past. Instead of averaging a fixed number of recent values, the exponential moving average updates the forecast step by step using a smoothing parameter.

$$\hat{y}_{t+1} = \alpha y_t + \alpha(1 - \alpha)y_{t-1} + \alpha(1 - \alpha)^2 y_{t-2} + \dots$$

α is the smoothing parameter ($0 < \alpha < 1$). $\hat{y}_{t+1}$ is the forecast at time t

The If α is large (close to 1) then the model reacts quickly to recent changes.

If α is small (close to 0) the model changes slowly and produces smoother forecasts.

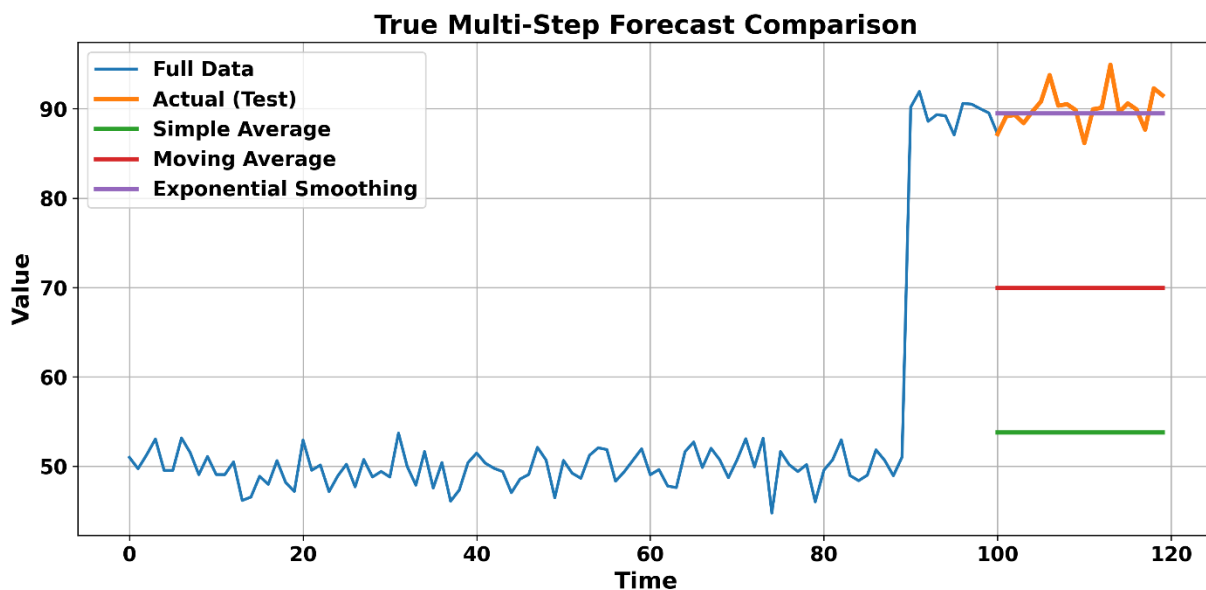
The above equation can also be written in recursive form as below

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha)\hat{y}_t$$

See that this is equivalent to previous equation. We can understand this equation The new forecast is a weighted combination of the most recent actual value y_t and the previous forecast $\hat{y}_t$

Exponential moving average is also known as Simple Exponential Smoothing (SES).

Example:



In this level shift example:

- **Simple Average** completely fails because it uses the entire past history, including the old low level. So it keeps predicting around the middle value and cannot adjust to the new higher level.
- **Moving Average** performs better because it uses only recent values. However, since it still gives equal weight to the last few observations, it adjusts more slowly after the shift.
- **Exponential Smoothing** performs the best because it gives more weight to recent observations. Once the level changes, it moves quickly toward the new level.

Holt's Linear Trend model

Simple exponential smoothing works well when the data fluctuates around a constant level. But many real-world time series do not behave like that. They show a steady increase or decrease over time — for example, airline passengers increasing every year or sales growing steadily.

If we use simple exponential smoothing on such data, it will always lag behind because it only estimates the **current level**, not the **rate of change**. It does not explicitly model growth or decline.

Holt's Linear Trend model solves this problem by modeling **two components**:

1. The current level of the series
2. The current trend (growth rate)

So instead of tracking only “where we are,” it also tracks “how fast we are moving.”

At every time step, we update:

- The estimated level at time t say L_t
- The estimated trend (slope) at time t say T_t

Then we forecast using both.

$$\hat{y}_{t+h} = L_t + hT_t$$

This is actually, Forecast = current level + ($h \times$ current trend). Here, the ‘current trend’ is nothing but the slope of the trend and h is the horizon (how many steps ahead you want to predict).

We need to now see how to obtain the the level and trend components.

Lets start with trend, trend is the slope in other words the difference between the levels. So the way we wrote previously the exponential smoothing recursive equation we can write the trend as

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Here you can see that the slope is $L_t - L_{t-1}$, and β is the smoothing parameter. This is very similar to previous exponential smoothing equation in which one is actual values and another is the estimate. We are obtaining the smooth estimate of the slope to handle sudden noisy jumps in the data which are temporary.

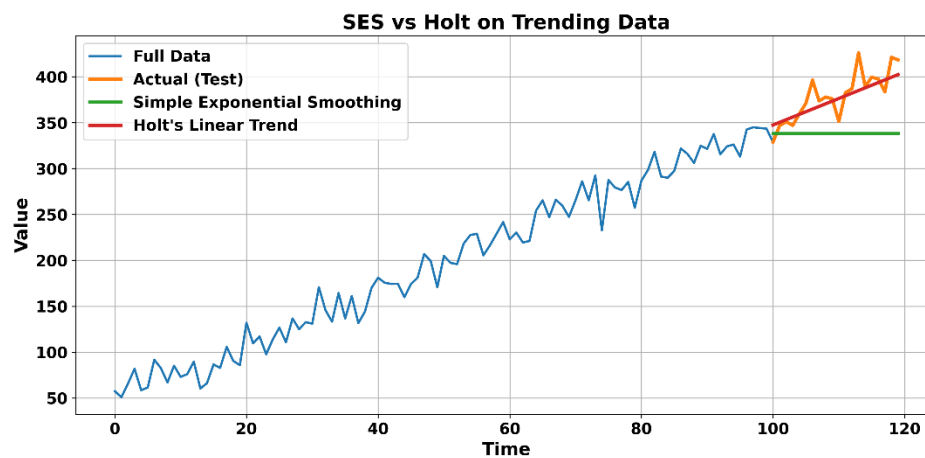
Now, we will write the equation for the level

$$L_t = \alpha y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

This is exactly same as the exponential smoothing with one actual value and previous estimate. The level at a point t is the sum of level and the trend one step before.

Note: α controls the reaction of level and β controls the reaction of the trend.

Example:



Damped trend

The trend not always continue infinitely. In real life:

- Growth slows down
- Markets saturate
- Demand stabilizes

How do we model this?

We use a method called damped trend. We introduce a new parameter ϕ

Instead of:

$$\hat{y}_{t+h} = L_t + hT_t$$

We use:

$$\hat{y}_{t+h} = L_t + T_t(\phi + \phi^2 + \phi^3 + \dots + \phi^h)$$

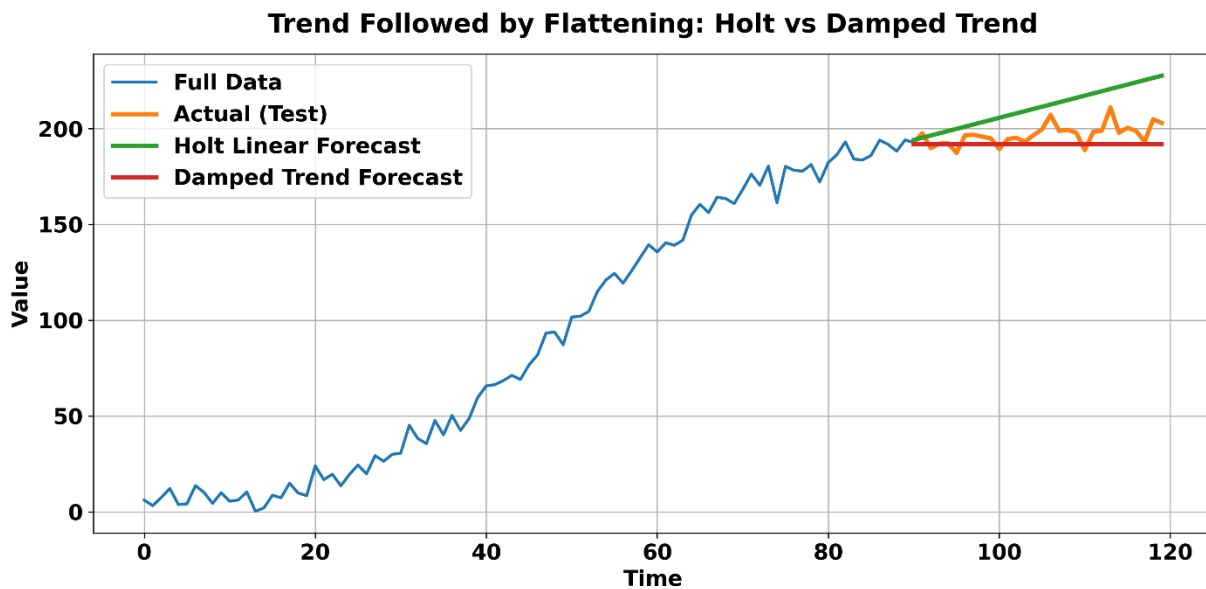
- $\phi = 1 \rightarrow$ No damping $\rightarrow$ same as Holt
- $\phi = 0.9 \rightarrow$ trend slowly decreases
- $\phi = 0.5 \rightarrow$ trend dies quickly

Level update:

$$L_t = \alpha y_t + (1 - \alpha)(L_{t-1} + \phi T_{t-1})$$

Trend update:

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)\phi T_{t-1}$$



We can see that damped model better forecasts the actual data.

Holt-Winters Method

Holt-Winters is used for time series with both trend and seasonality.

It extends:

- Simple Exponential Smoothing → handles level only
- Holt's Method → handles level + trend
- Holt-Winters → handles level + trend + seasonality

At any time t , data y_t is assumed to be made of:

(i) Additive seasonal component (Seasonality is constant size) : $\text{Data} = \text{Level} + \text{Trend} + \text{Seasonality}$
or

(ii) Multiplicative seasonal component (Seasonality grows with level):
 $\text{Data} = (\text{Level} + \text{Trend}) \times \text{Seasonality}$

At each time step we estimate:

- $L_t \rightarrow$ Level (baseline value)
- $T_t \rightarrow$ Trend (slope)
- $S_t \rightarrow$ Seasonal component
- $\alpha, \beta, \gamma \in (0, 1) \rightarrow$ smoothing parameters
- $m \rightarrow$ season length (e.g., 12 for monthly data)

The forecast equation is

$$\hat{y}_{t+h} = L_t + hT_t + S_{t+h-mk}$$

Where k is used to get the season index within the available past data $k = \text{int}((h-1)/m)$

Step 1: Update Level

$$L_t = \alpha(y_t - S_{t-m}) + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Same as before but only difference is subtracting the seasonal component. Assuming data is sum of level and seasonality component with trend component already in the trend part.

Step 2: Update Trend

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Same as before

Step 3: Update Seasonality

$$S_t = \gamma(y_t - L_t) + (1 - \gamma)S_{t-m}$$

Same recursion with actual value and the estimated value. Actual value is y_t as we have assumed level already contains the trend part.

For multiplicative seasonality component

Forecast

$$\hat{y}_{t+h} = (L_t + hT_t) \times S_{t+h-mk}$$

Level update

$$L_t = \alpha \frac{y_t}{S_{t-m}} + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Actual level obtained by dividing the seasonality

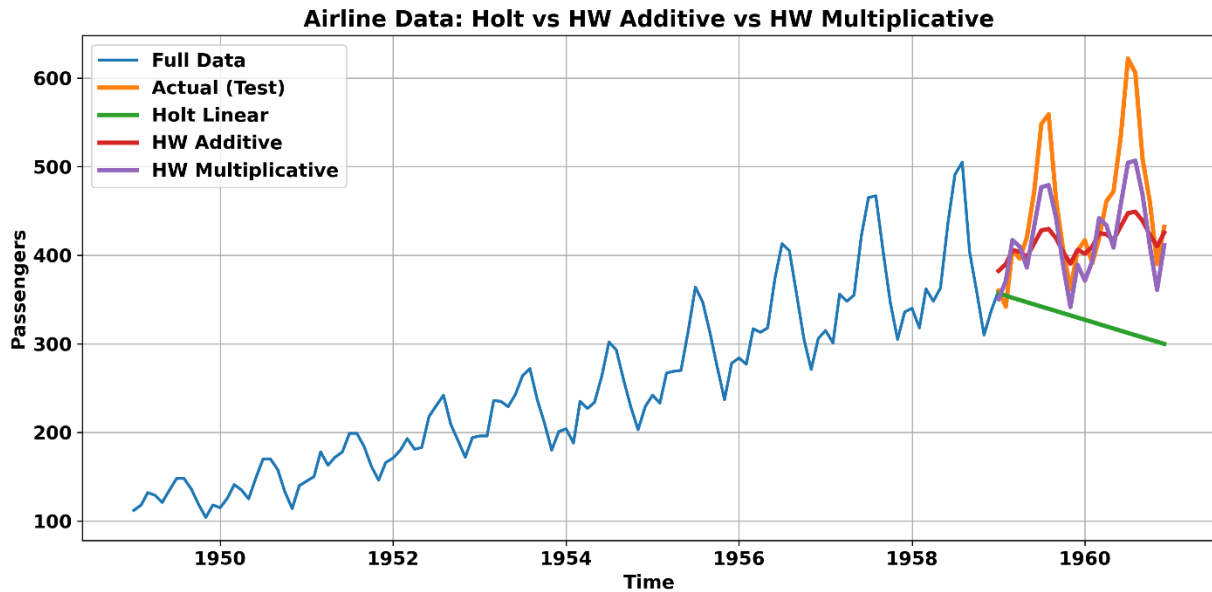
Trend Update

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Seasonality update

$$S_t = \gamma \frac{y_t}{L_t} + (1 - \gamma)S_{t-m}$$

Actual seasonality obtained by dividing the level



ETS Models (Error, trend, seasonality models)

These are same as exponential smoothing models but they also provide the intervals for the error. They are more statistical where exponential methods are called point forecasting methods.

Same as before, ETS models describe a time series using three simple components:

$$y_t = \text{Level} + \text{Trend} + \text{Seasonality} + \text{Error}$$

They are called state space models because they update internal states (level, trend, seasonality). These states evolve over time but are not visible as we see only the time series data and not these states. These states we estimate!!!

Lets understand the ETS models using the simple exponential smoothing or level estimation

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t$$

This equation can also be written assuming this method only estimates the level

$$\hat{y}_{t+h} = L_t$$

where

$$L_t = \alpha y_t + (1 - \alpha) L_{t-1}$$

Lets focus on interval part now. Prediction is never perfect. There is always **uncertainty**.

Instead of saying: Prediction = 105

we say Prediction = $105 \pm \text{uncertainty}$

Example:

Prediction = 105

Interval = [100 , 110]

Meaning: There is 95% probability the real value will fall inside this range.

Let derive the mathematically the error intervals

So the **forecast error** is

$$e_t = y_t - \hat{y}_t$$

Substitute $\hat{y}_t = L_{t-1}$:

$$e_t = y_t - L_{t-1}$$

(Recollect the previous estimate is the L_{t-1})

$$L_t = \alpha y_t + (1 - \alpha)L_{t-1}$$

Expand the second term.

$$L_t = \alpha y_t + L_{t-1} - \alpha L_{t-1}$$

Rearrange terms.

$$L_t = L_{t-1} + \alpha(y_t - L_{t-1})$$

So using the error equation

$$L_t = L_{t-1} + \alpha e_t$$

Similarly using

$$e_t = y_t - L_{t-1}$$

And substituting $t=t+1$ we can get

$$y_{t+1} = L_t + e_{t+1}$$

$$y_{t+2} = L_{t+1} + e_{t+2}$$

We know

$$L_{t+1} = L_t + \alpha e_{t+1}$$

So

$$y_{t+2} = L_t + \alpha e_{t+1} + e_{t+2}$$

We can write

$$y_{t+h} = L_t + \alpha e_{t+1} + \alpha e_{t+2} + \cdots + \alpha e_{t+h-1} + e_{t+h}$$

$$y_{t+h} = L_t + \alpha \sum_{i=1}^{h-1} e_{t+i} + e_{t+h}$$

$$e_{t+h|t} = y_{t+h} - \hat{y}_{t+h}$$

$$e_{t+h|t} = \alpha \sum_{i=1}^{h-1} e_{t+i} + e_{t+h}$$

$$\text{Var}(e_t) = \sigma^2 \quad \text{assuming error to be } e_t \sim N(0, \sigma^2)$$

We use the basic rule

$$\text{Var}(aX) = a^2 \text{Var}(X)$$

$$\begin{aligned} \text{Var}(e_{t+h|t}) &= \alpha^2 \sum_{i=1}^{h-1} \text{Var}(e_{t+i}) + \text{Var}(e_{t+h}) \\ &= \alpha^2(h-1)\sigma^2 + \sigma^2 \end{aligned}$$

$$\text{Var}(e_{t+h|t}) = \sigma^2[1 + (h-1)\alpha^2]$$

Now the prediction interval is given as below

$$\begin{aligned} &\hat{y}_{t+h} \pm z\sqrt{\text{Var}} \\ &= L_t \pm z\sigma\sqrt{1 + (h-1)\alpha^2} \end{aligned}$$

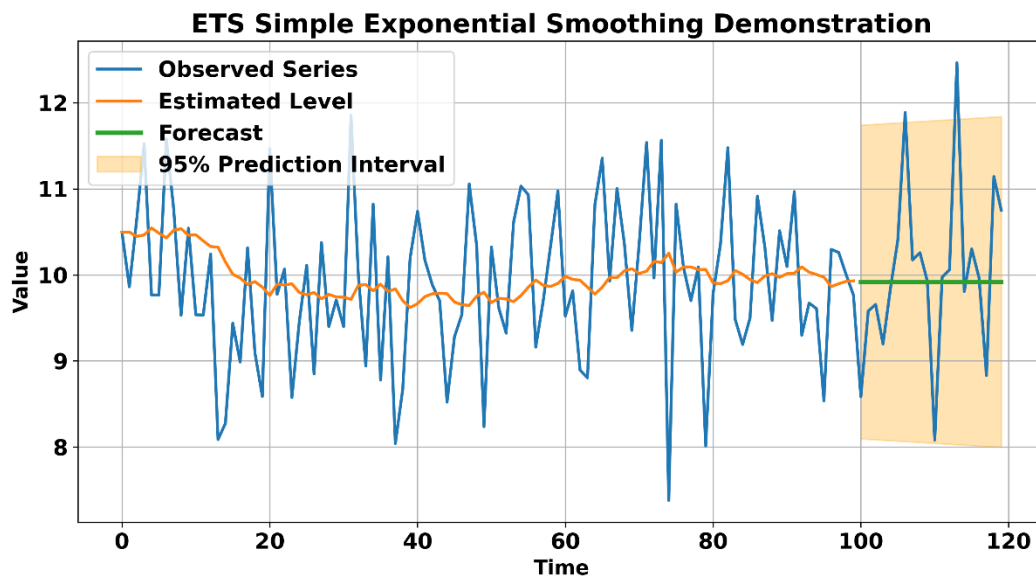
z = value from the normal distribution => 1.96 for 95% interval of normal distribution

(68% of values lie within $\pm 1\sigma$, 95% lie within $\pm 1.96\sigma$, 99% lie within $\pm 2.58\sigma$)

SES uncertainty increases linearly with horizon.

But the increase depends on α .

- small $\alpha \rightarrow$ stable forecasts
- large $\alpha \rightarrow$ rapidly increasing uncertainty



Shaded band $\rightarrow$ range where the true value is likely to appear assuming our error distribution to be Normal. There is about a 95% chance that the next observed value will lie between 8.5 and 11.5.

We can derive such expressions for all the exponential-based methods we studied.

Autocorrelation function

Autocorrelation measures how similar a time series is to itself after shifting it in time.

Example:

Suppose you have onion prices every day.

- If today's price is high and tomorrow's price is also usually high, the series has high autocorrelation at lag 1.
- If today's price is related to the price two days ago, then there is autocorrelation at lag 2.

The autocorrelation at lag k is

$$\rho_k = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

where

- y_t = value at time t
- $\bar{y}$ = mean of the series
- k = lag
- ρ_k = autocorrelation at lag k

Value of ρ_k : 1=> strong positive relationship, 0=> No relationship, -1=> strong negative relationship

Partial Autocorrelation Function (PACF)

PACF measures the direct relationship between a time series value and its lagged value after removing the effect of intermediate lags.

In a time series values are often correlated with previous observations. However, some of these correlations arise indirectly through other lagged values.

Example:

If $y_{t-2} \rightarrow y_{t-1} \rightarrow y_t$

then y_{t-2} may appear correlated to y_t , even if there is no direct dependence.

The PACF removes the influence of intermediate lags and measures only the direct correlation between y_t and y_{t-k}

The formula for calculating the PACF(k) represented as $\phi_{k,k}$

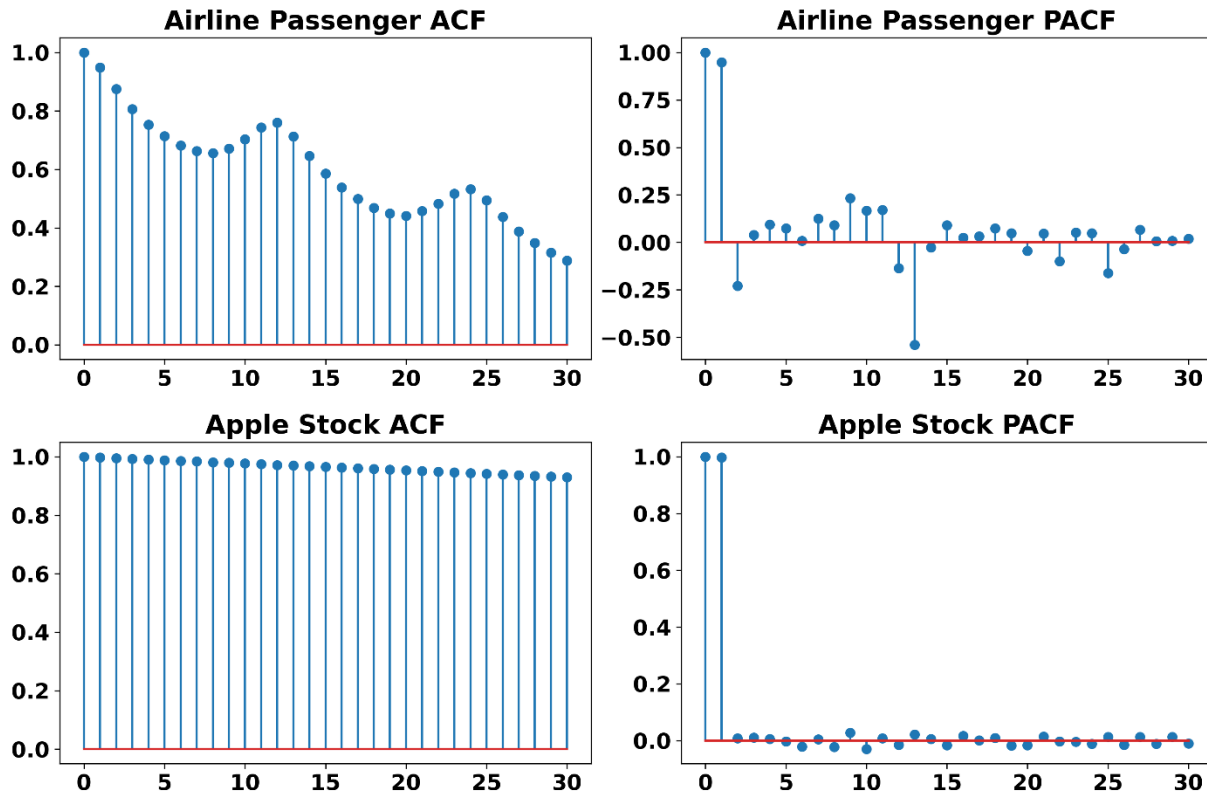
$$\phi_{k,k} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_j}$$

$$PACF = \frac{\text{total correlation} - \text{indirect correlation}}{\text{remaining variance}}$$

(This is similar to

- $\phi_{k,k}$ – Partial autocorrelation at lag k ; the direct correlation between the current value and the value k steps in the past after removing intermediate effects.
- ρ_k – Autocorrelation at lag k ; the total correlation between the current value and the value k steps earlier.
- $\phi_{k-1,j}$ – A coefficient computed in the previous step that represents how much lag j contributes when removing intermediate correlations.
- $\sum_{j=1}^{k-1} \phi_{k-1,j} \rho_{k-j}$ – The part of the correlation at lag k that occurs indirectly through intermediate lags.
- **Numerator** $\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_{k-j}$ – Total correlation minus indirect correlations, leaving only the direct relationship.
- $\sum_{j=1}^{k-1} \phi_{k-1,j} \rho_j$ – A measure of how much correlation is already explained by the intermediate lags.
- **Denominator** $1 - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_j$ – A normalization term that adjusts the result so the partial correlation stays between -1 and 1 .

Figure shows the ACF and PACF for two examples. We can see that there are so many peaks in the ACF but PACF have only few peaks. This indicate there are lot of indirect correlations which may not be relevant for time series forecasting. PACF helps in identifying those lags which are very important for current sample.



Stationary time series

A stationary time series is a series whose statistical behavior does not change over time.

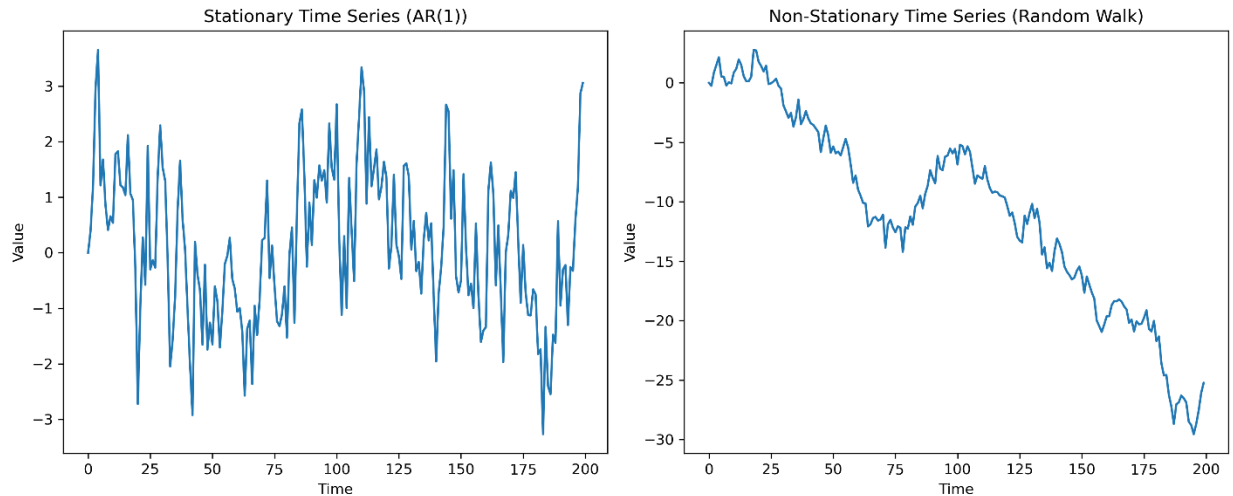
In simple terms, the series behaves similar everywhere in time.

The following properties stay constant:

1. Mean (average level)
2. Variance (amount of fluctuations)
3. Correlation between points

Example are shown in the figures

You can see that the mean and variance are almost constant for stationary where as for the non-stationary case it is changing over time. Even though the mean and variance are constant but not exactly same and thus even the first graph is not strictly stationary but is called weakly stationary.



Weak Stationarity

Most models assume weak stationarity.

Conditions:

1. Constant mean
2. Constant variance
3. Autocorrelation depends only on lag depends only on k, not time.

Difference operation to make a process stationary

The difference operation removes this trend and makes the series stationary. Differencing means computing the change between consecutive observations.

$$y'_t = y_t - y_{t-1}$$

We can do multiple difference till the series becomes stationary

Testing for Stationarity

Unit root test is a statistical test used to determine whether a time series has a unit root, which is a characteristic of a non-stationary time series.

Consider an AR(1) model

$$y_t = \phi y_{t-1} + \epsilon_t$$

If

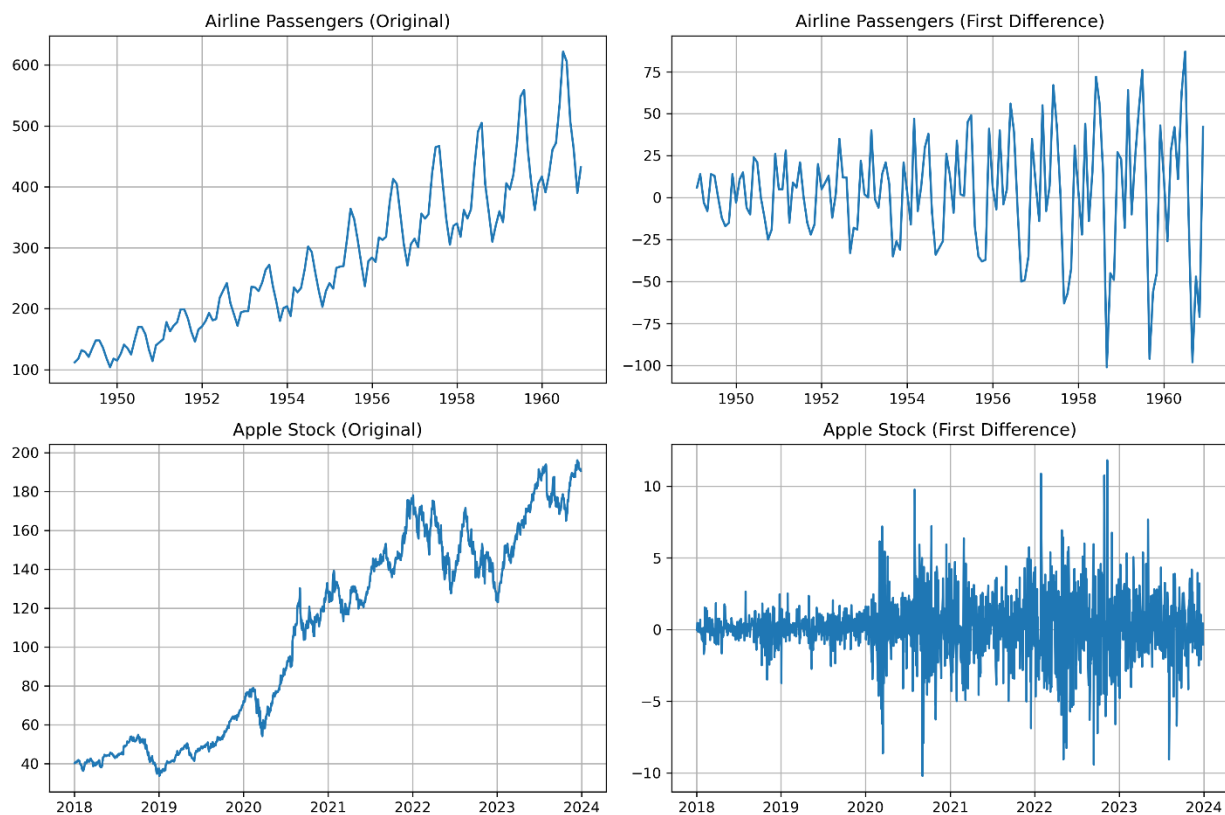
$$\phi = 1$$

then

$$y_t = y_{t-1} + \epsilon_t$$

This series is called random walk and is non-stationary. So the test checks whether the coefficient is equal to 1 (unit root test)

Augmented Dickey-Fuller (ADF) test is used to check for this unit root existence. It gives a p value. If p values is less than 0.05 then stationary otherwise non-stationary.



ADF Test: Airline Original

ADF Statistic : 0.8153688792060441

p-value : 0.9918802434376409

Result : Non-Stationary

ADF Test: Airline First Difference

ADF Statistic : -2.8292668241700034

p-value : 0.05421329028382497

Result : Non-Stationary

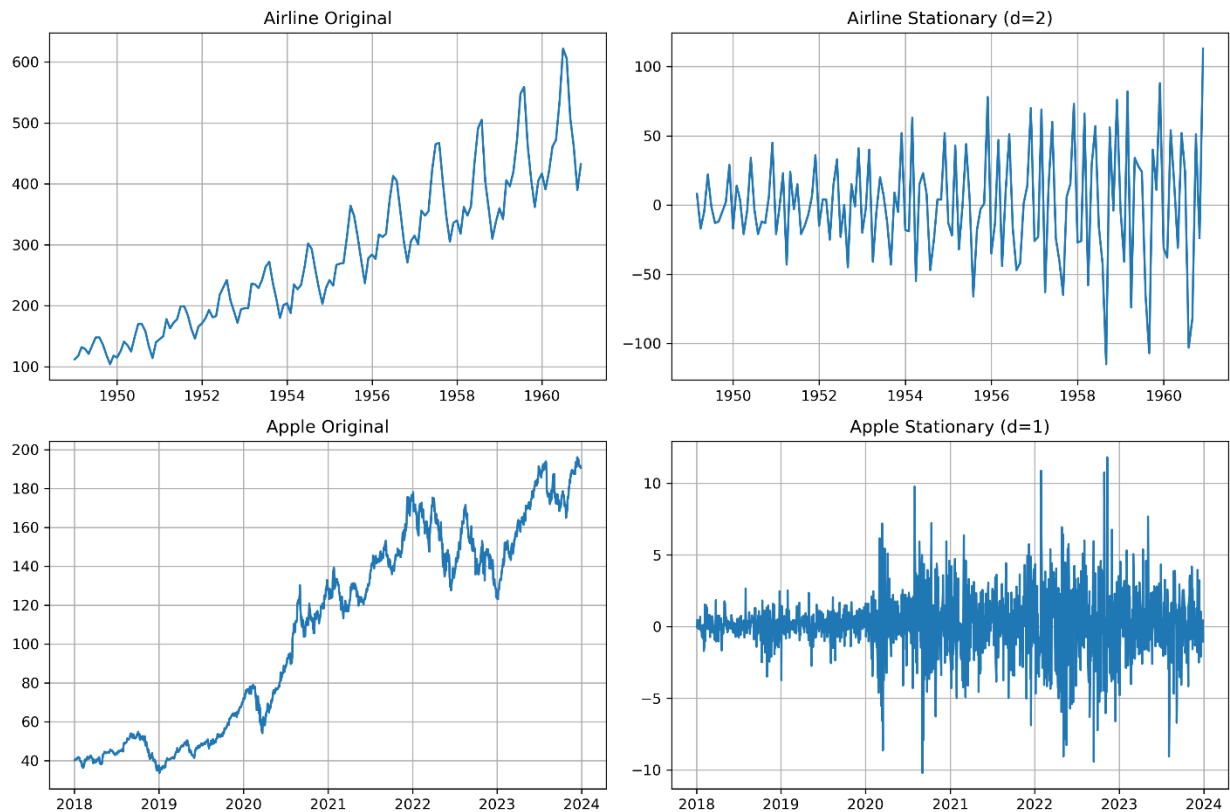
ADF Test: Apple Stock Original

ADF Statistic : -0.5118334067397655

p-value : 0.8896865266495615
Result : Non-Stationary

ADF Test: Apple Stock First Difference
ADF Statistic : -40.349901819111395
p-value : 0.0
Result : Stationary

Do the second difference to make airline data stationary



=====
Processing Airline Passengers
=====

ADF Test for Airline Passengers Original
ADF Statistic : 0.8153688792060441
p-value : 0.9918802434376409
Result : NON-STATIONARY

ADF Test for Airline Passengers First Difference
ADF Statistic : -2.8292668241700034
p-value : 0.05421329028382497
Result : NON-STATIONARY

ADF Test for Airline Passengers Second Difference
ADF Statistic : -16.38423154246852
p-value : 2.732891850014085e-29
Result : STATIONARY

Series became stationary after second differencing.

```
=====
Processing Apple Stock
=====
```

```
ADF Test for Apple Stock Original
ADF Statistic : -0.5118334067397655
p-value      : 0.8896865266495615
Result       : NON-STATIONARY
```

```
ADF Test for Apple Stock First Difference
ADF Statistic : -40.349901819111395
p-value      : 0.0
Result       : STATIONARY
```

Autoregressive Model (AR)

The current value depends on previous values of the signal. In simple words: The present is a weighted combination of the past. For using past p values (order of AR) we can write

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$$

Obtain the optimal parameters $c, \phi_1, \phi_2, \phi_3, \dots, \phi_p$ using the past data

Forecast is

One step forecast

$$\hat{y}_{t+1} = c + \phi_1 y_t + \phi_2 y_{t-1} + \dots + \phi_p y_{t+1-p}$$

Multistep forecast

$$\hat{y}_{t+h|t} = c + \phi_1 \hat{y}_{t+h-1|t} + \phi_2 \hat{y}_{t+h-2|t} + \dots + \phi_p \hat{y}_{t+h-p|t}$$

The Autoregressive (AR) model is important in time series analysis because it captures the dependence of the current value on several past values of the series, allowing the model to represent the internal memory and temporal correlations of the data. By expressing the present observation as a weighted sum of previous observations, AR models can describe dynamic behaviors such as gradual changes and oscillatory patterns that often occur in real-world signals. Compared to simple exponential smoothing, which mainly relies on the most recent estimate with exponentially decreasing weights, AR models explicitly use multiple past observations with optimal weights to better capture the underlying autocorrelation structure of the time series, leading to more accurate modeling of stationary processes.

Note that for autoregressive method to be reliable, the time series should be stationary (weak sense) otherwise if there is a trend and the variance varies then the model cannot reliably forecast the future values.

How to choose the order p of the AR model?

P represents how many past values we use. Choose the p when there is a sudden drop in the PACF values.

How to obtain the parameters of the AR model?

There are many methods. Least square solution is one of them.

The model predicts

$$\hat{y}_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p}$$

The **prediction error (residual)** is

$$e_t = y_t - \hat{y}_t$$

Least squares chooses parameters that **minimize the total squared error**

$$S = \sum_{t=p+1}^T e_t^2$$

$$S = \sum_{t=p+1}^T (y_t - c - \phi_1 y_{t-1} - \dots - \phi_p y_{t-p})^2$$

We will minimize this error to obtain the parameters.

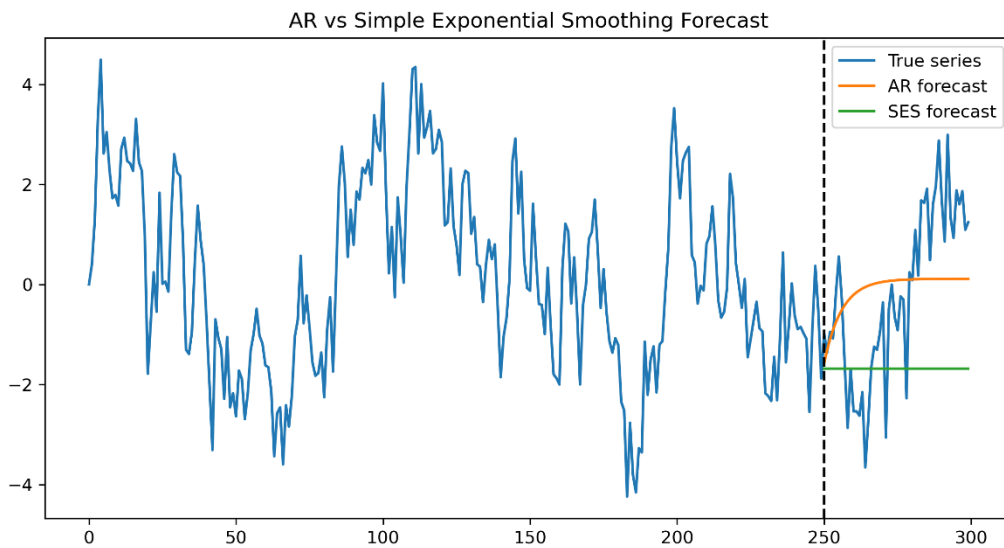
We will use matrix representation. Consider the samples $y_1, y_2, \dots, y_T$

$$Y = \begin{bmatrix} y_{p+1} \\ y_{p+2} \\ \vdots \\ y_T \end{bmatrix} \quad X = \begin{bmatrix} 1 & y_p & y_{p-1} & \dots & y_1 \\ 1 & y_{p+1} & y_p & \dots & y_2 \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & y_{T-1} & y_{T-2} & \dots & y_{T-p} \end{bmatrix} \quad \beta = \begin{bmatrix} c \\ \phi_1 \\ \phi_2 \\ \vdots \\ \phi_p \end{bmatrix}$$

$$Y = X\beta + \varepsilon$$

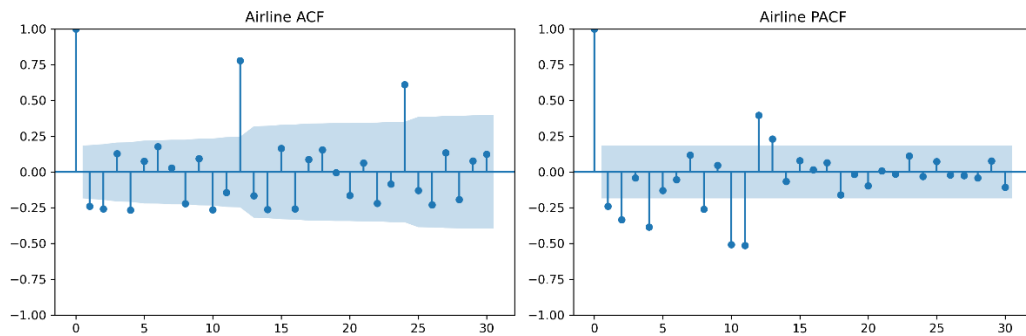
Using simple least square solution we can obtain $\hat{\beta} = (X^T X)^{-1} X^T Y$

Comparing AR and simple exponential smoothing

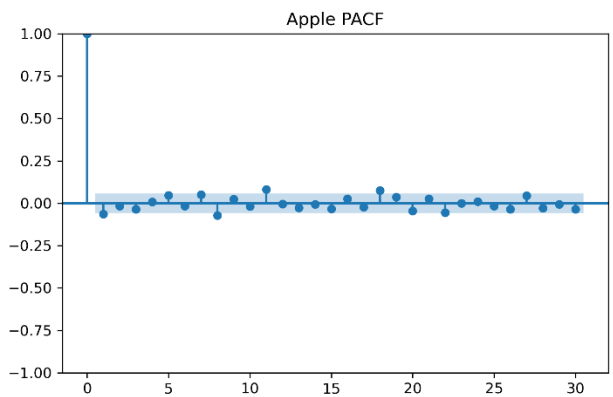
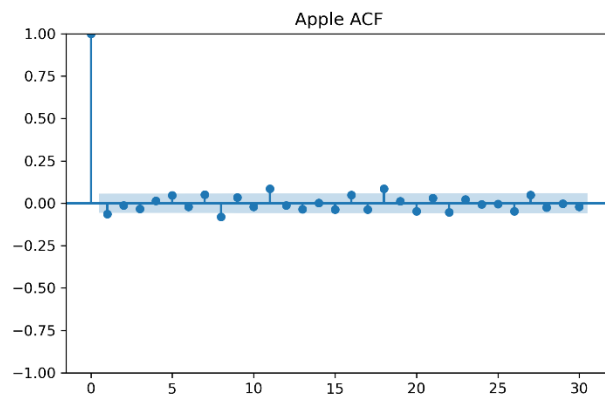
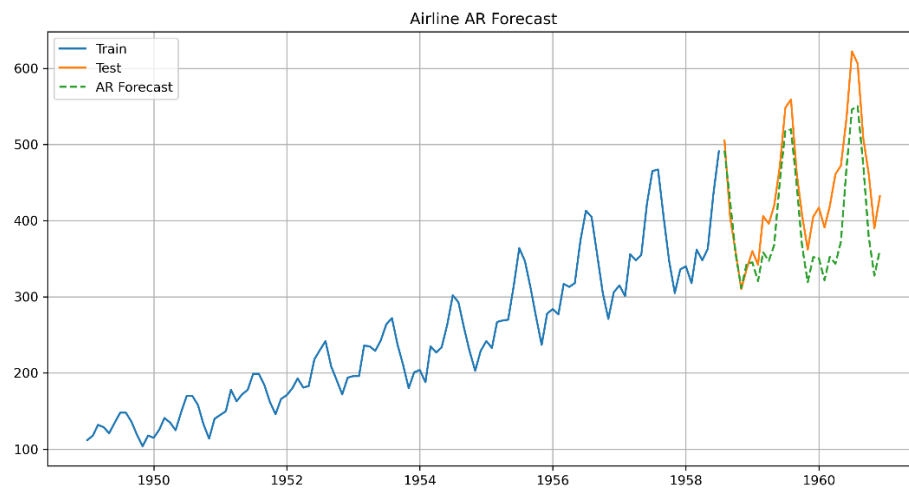


We can see that the time series data is stationary. We can see that AR model better estimates the future values compared to simple exponential smoothing method as the AR model better captures the relationship of the future sample with the past values.

Applying AR method on the airline passenger data. For this we know that data is not stationary so we will do the difference operator before applying the AR model and then do reverse to compensate.



Choosing $p=12$



Choosing $p=1$



Moving Average Method

A Moving Average (MA) model expresses the current value of a time series as a linear combination of past random shocks (errors).

An MA model of order q is written as:

$$y_t = \mu + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + \dots + \theta_q\epsilon_{t-q}$$

where

- y_t : value of the time series at time t
- μ : mean of the series
- ϵ_t : Error at time t
- θ_i : MA parameters
- q : order of the MA model

The **error term** represents the **unpredictable shock** at that time.

It is defined as

$$\epsilon_t = y_t - \hat{y}_t$$

where

- y_t = actual value
- $\hat{y}_t$ = model prediction

So the error is simply the **difference between actual value and predicted value**.

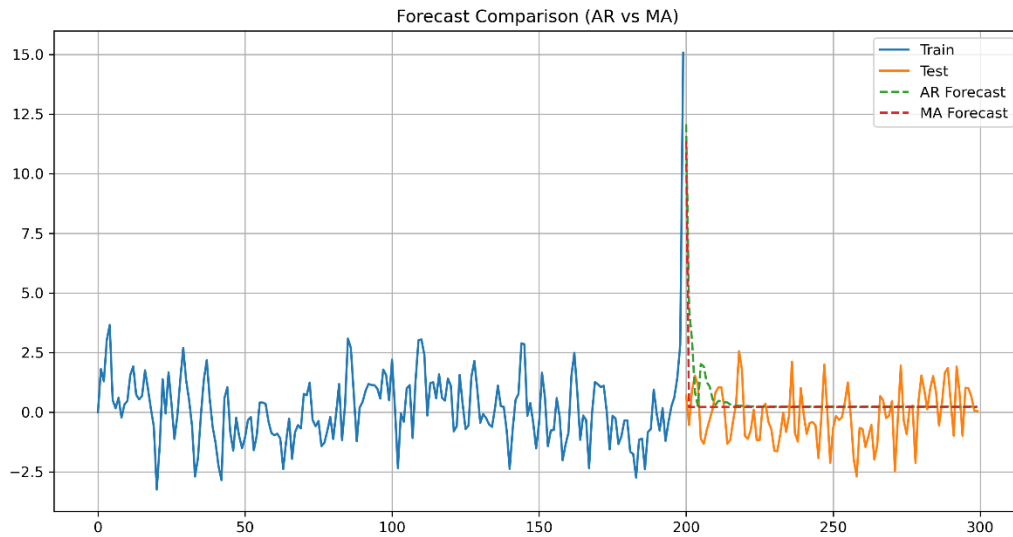
The present value depends on current shock + past shocks. It performs best when the effect of shocks dies quickly.

Forecast

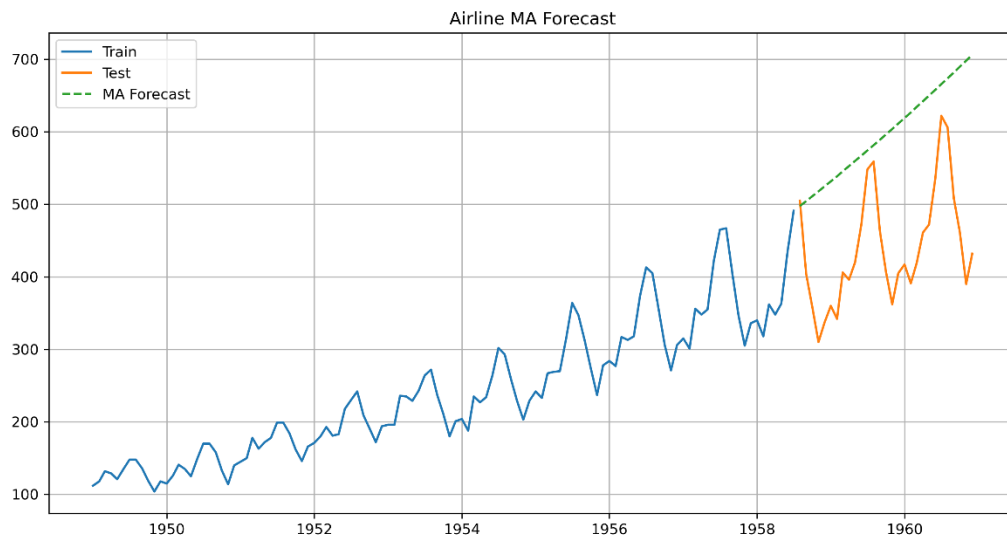
$$\hat{y}_{t+1} = \mu + \theta_1\epsilon_t + \theta_2\epsilon_{t-1} + \dots + \theta_q\epsilon_{t-q+1}$$

When we do not know the actual values in recursive estimation, we treat them as zero and just use AR terms.

The order is determined using the when the ACF cuts off quickly.



AR assumes that the large change in the series will continue into the future, so it slowly carries that effect forward. In contrast, the MA model recognizes that the large change is just a temporary shock. As a result, the MA forecast quickly returns to the average value, while the AR forecast decreases gradually over time. This highlights the importance of the MA model: it is designed to capture short-lived shocks or disturbances that affect a series only for a brief period. Therefore, MA models work well for processes where unexpected events cause temporary fluctuations but their effects disappear quickly.



We used the order 1 for the MA. For airline passenger we do not see the seasonal variations as MA cannot calculate errors for the future samples and hence it does not have the seasonal variations that we can see using AR method. For APPLE stock, plots of AR and MA look the same.

ARMA models

An ARMA(p, q) model assumes that the current value of a time series depends on:

1. Past values of the series $\rightarrow$ AR part

2. Past forecast errors (shocks) → MA part

It can be written as

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$$

Where:

- y_t = value at time t
- c = constant
- p = AR order
- q = MA order
- ϕ_i = AR coefficients
- θ_j = MA coefficients
- ϵ_t = white noise error

One step forecast for ARMA(1,1) can be written as

$$\hat{y}_{t+1} = c + \phi_1 y_t + \theta_1 \epsilon_t$$

ARIMA models

The process of converting non-stationary series to stationary series using difference is represented by the letter 'I' which means integrated.

ARIMA(p,d,q): d represents the number of times we applied the difference.

Some notations are given below

First difference:

$$z_t = y_t - y_{t-1}$$

Second difference:

$$z_t = (y_t - y_{t-1}) - (y_{t-1} - y_{t-2})$$

General form:

$$z_t = (1 - B)^d y_t$$

where

- B = backshift operator
- $By_t = y_{t-1}$

The ARIMA equation can be written as

$$z_t = c + \sum_{i=1}^p \phi_i z_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$$

where

$$z_t = (1 - B)^d y_t$$

Using the B terms the ARIMA equation can be written as

$$(1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p)(1 - B)^d y_t = c + (1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q) \epsilon_t$$

See $(1-B)^d$ represents the d times difference is applied on y_t

SARIMA model

Even though the ARMA models handle the seasonal part but are not very good at it. We use seasonal ARIMA models to better approximate these series.

Seasonal difference: It removes repeating seasonal patterns.

$$z_t = y_t - y_{t-s}$$

Example (monthly data):

$$z_t = y_t - y_{t-12}$$

SARIMA(p,d,q)(P,D,Q)s

P,d,q represent the order for ARIMA

(P,D,Q)s represents the orders for the seasonal part.

Using backshift notation:

$$\Phi(B^s)\phi(B)(1-B)^d(1-B^s)^D y_t = \Theta(B^s)\theta(B)\epsilon_t$$

Where

Non-seasonal parts

$$\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$$

$$\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$$

Seasonal parts

$$\Phi(B^s) = 1 - \Phi_1 B^s - \dots - \Phi_P B^{Ps}$$

$$\Theta(B^s) = 1 + \Theta_1 B^s + \dots + \Theta_Q B^{Qs}$$

Lets taken an example of SARIMA (1,1,1)(1,1,1) assume the seasonal period as 12 for monthly sample

$$(1 - \Phi_1 B^{12})(1 - \phi_1 B)(1 - B)(1 - B^{12})y_t = (1 + \Theta_1 B^{12})(1 + \theta_1 B)\epsilon_t$$

We can first expand all the terms on the left side of y_t and then operate the B operator.

Let's start with y_t

$$z_t = (1 - B^2) y_t$$

step 1: $z_t = y_t - y_{t-2}$

$$z'_t = (1 - B) z_t$$

step 2: $z'_t = z_t - z_{t-1} = y_t - y_{t-2} - y_{t-1} + y_{t-3}$

step 3: $(1 - \phi_1 B) z'_t = z'_t - \phi_1 z'_{t-1} \Rightarrow$ let's call this z''_t

$$\begin{aligned} z''_t &= y_t - y_{t-2} - y_{t-1} + y_{t-3} - \phi_1 [y_{t-1} - y_{t-3} - y_{t-2} + y_{t-4}] \\ &= y_t - (1 + \phi_1) y_{t-1} - \phi_1 y_{t-2} - y_{t-2} + (1 - \phi_1) y_{t-3} - \phi_1 y_{t-4} \end{aligned}$$

step 4: $(1 - \phi_1 B^2) z''_t = z''_t - \phi_1 z''_{t-2}$

LHS = $z''_t - \phi_1 z''_{t-2} \Rightarrow$ we can substitute z''_t from step 3

To keep it simple we can see that LHS has only y_t & its delayed terms

$$\text{RHS} = (1 + \theta B^2) (1 + \theta B) \varepsilon_t$$

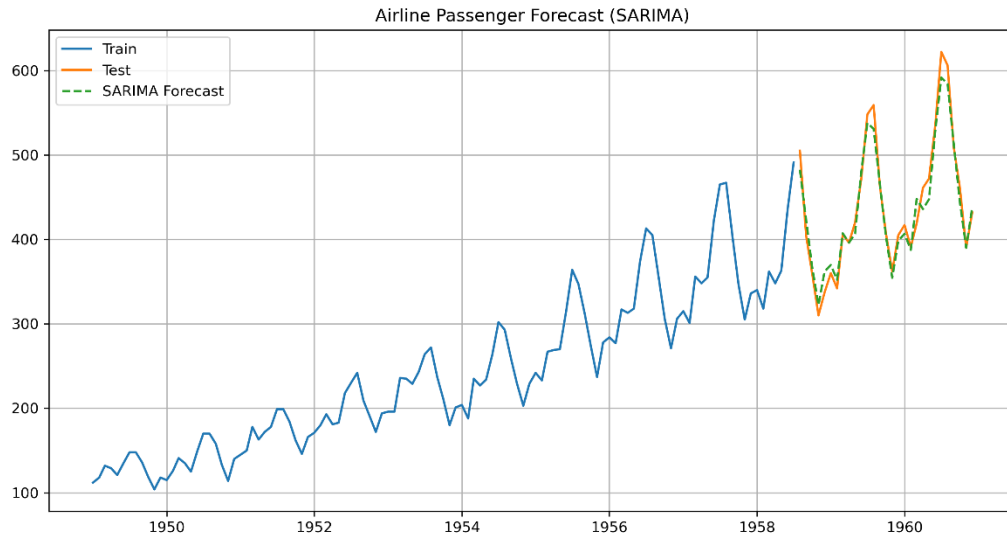
$$= (1 + \theta B^2) (\varepsilon_t + \theta \varepsilon_{t-1})$$

$$= \varepsilon_t + \theta \varepsilon_{t-1} + \theta \varepsilon_{t-2} + \theta \theta \varepsilon_{t-3}$$

Interpretation of SARIMA

We will apply the normal difference and seasonal difference to make the series stationary in the normal sense and seasonal sense as well.

After the sequence is stationary we approximate the series as sum of normal lag terms and seasonal lag terms assuming y_t is influenced by the normal lags as well as previous seasonal lags.



We can see that SARIMA gives the best results among all the model as it models the seasonality best.

SARIMAX (SARIMA Exogenous variables)

It adds **external variables**.

X = exogenous variables

These are variables that influence the target series. The forecasting improves due to the addition of these variables

Eg: We can use macroeconomic indicators as exogenous variables during stock price prediction

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \sum_{k=1}^K \beta_k X_{k,t} + \epsilon_t$$

Where

- y_{t-i} = past values
- ϵ_{t-j} = past shocks
- $X_{k,t}$ = external variables

Vector autoregression (VAR) models

Vector autoregression predicts multiple variables together, using the past of all variables.

Example variables:

- A_t = apple price
- B_t = banana price

VAR(1) model:

$$A_t = c_1 + a_{11}A_{t-1} + a_{12}B_{t-1} + \epsilon_{1t}$$

$$B_t = c_2 + a_{21}A_{t-1} + a_{22}B_{t-1} + \epsilon_{2t}$$

Apple price today depends on:

- past apple price
- past banana price

Banana price today depends on:

- past apple price
- past banana price

We combine variables into a **vector**.

$$Y_t = \begin{bmatrix} A_t \\ B_t \end{bmatrix}$$

The model becomes

$$Y_t = c + A_1 Y_{t-1} + \epsilon_t$$

Where

$$A_1 = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

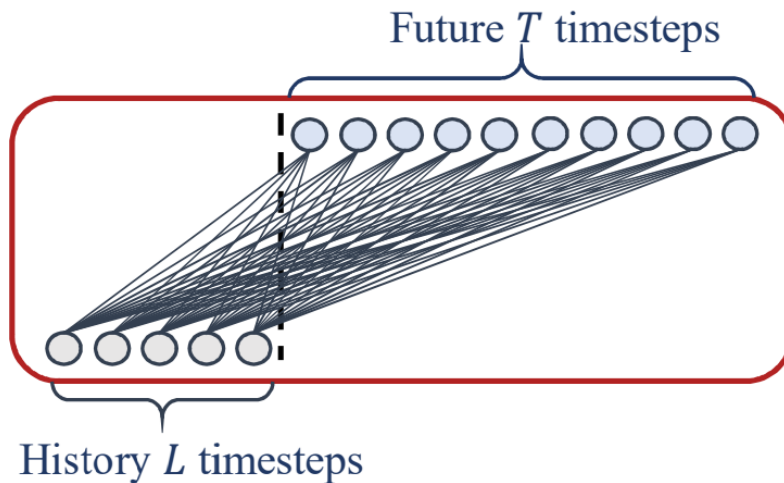
The vector representation is the reason the model is termed as Vector Autoregression

PLEASE USE THE SLIDES FOR BASICS OF ML and DECISION TREE MODELS

DLinear and NLinear models (From Are Transformers Effective for Time Series Forecasting? By Zeng et al. 2022, 4889 citations)

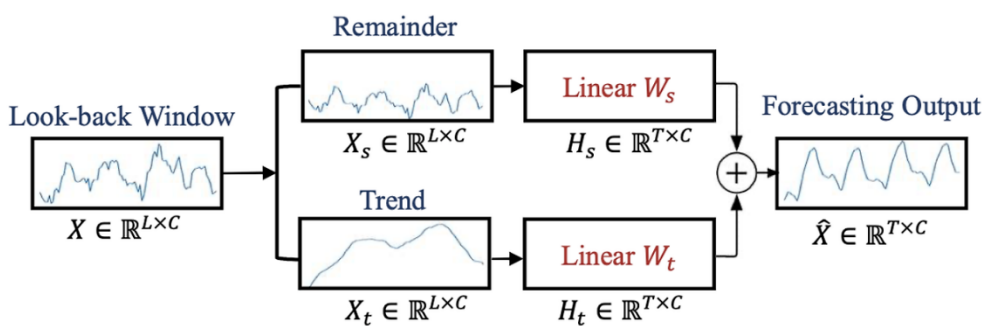
Both these models use weighted sum (known as **linear layers**)

Linear Layer



Linear layer directly regresses historical time series for future prediction via a weighted sum operation. In simple words “Take past values $\rightarrow$ multiply by weights $\rightarrow$ sum $\rightarrow$ predict future”

DLinear (Decomposition Linear Model)



DLinear first **splits the time series into two parts**:

1. **Trend** $\rightarrow$ slow, smooth changes
2. **Remainder** (seasonal) $\rightarrow$ Remaining part after removing the trend

Then it learns each part **separately using linear models**, and adds them back.

It first decomposes a raw data input into a trend component by a moving average kernel and a remainder (seasonal) component. Then, two one-layer linear layers are applied to each component, and we sum up the two features to get the final prediction.

NLinear model

If we normalize test data using the mean and variance from the training data, it may not work well if the test data has a different distribution. In such cases, the test data may not be centered around zero like the training data, causing a mismatch. As a result, the model may produce predictions that do not align with the true values, leading to larger errors.

Method: Instead of decomposing, it removes the last value (shift) before learning.

Step-by-step

Step 1: Normalize by last value

$$x'(t) = x(t) - x(T)$$

Where:

- $x(T)$ = last observed value
-

Step 2: Apply linear model

$$\hat{y}' = W \cdot x'$$

Step 3: Add back the last value

$$\hat{y} = \hat{y}' + x(T)$$

Subtracting the last value works because it represents the most recent level of the series, which is the best reference point for predicting the immediate future. Time series data often changes over time (trend or shifts), so older values may not reflect the current state. By subtracting the last value, the model learns relative changes from the present, which are usually more stable and easier to predict.

Standard Time series datasets: <https://github.com/juyongjiang/TimeSeriesDatasets>

Brief Introduction of Datasets (from the github page)

- [ETT \(Electricity Transformer Temperature\)](#): This dataset consists of two hourly-level datasets (ETTh) and two 15-minute-level datasets (ETTm). Each of them contains one oil and six load features of electricity transformers from July 2016 to July 2018.
- [Traffic](#): This dataset from the California Department of Transportation describes the road occupancy rates between 0 and 1. It contains the hourly data recorded by the sensors of San Francisco Bay area freeways from 2015.01 to 2016.12.
- [Electricity](#): This dataset collects the hourly electricity consumption in KWh of 321 clients from 2012 to 2014. Note that, the raw dataset records every 15 minutes from 2011 to 2014 while some dimensions of data are equal to 0. Thus, the records in 2011 are removed.
- [Exchange-Rate](#): This dataset collects the daily exchange rates of 8 countries, including Australia, British, Canada, Switzerland, China, Japan, New Zealand and Singapore, from 1990.01 to 2016.12.
- [Weather](#): This dataset includes 21 indicators of weather, such as air temperature, and humidity. Its data is recorded every 10 min for 2020 in Germany.
- [ILI \(Influenza-like Illness\)](#): This dataset describes the ratio of patients seen with influenza-like illness and the total number of the patients. It includes the weekly data from the Centers for Disease Control and Prevention of the United States from 2002 to 2021.

Performance comparison of the DLinear and NLinear models with other state of the art models

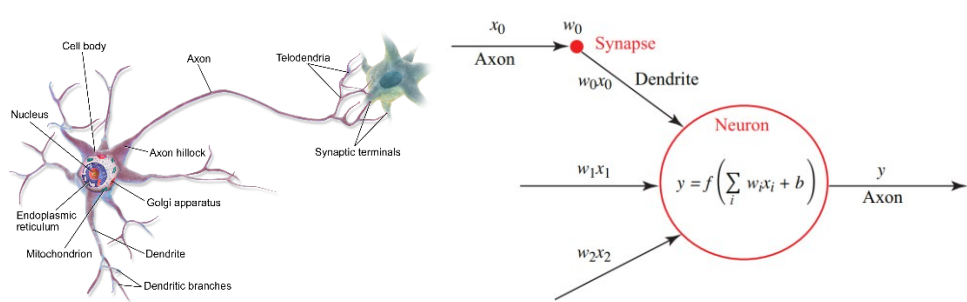
Methods		IMP.	Linear*		NLinear*		DLinear*		FEDformer		Autoformer		Informer		Pyraformer*		LogTrans		Repeat*	
Metric		MSE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Electricity	96	27.40%	0.140	0.237	0.141	0.237	0.140	0.237	<u>0.193</u>	<u>0.308</u>	0.201	0.317	0.274	0.368	0.386	0.449	0.258	0.357	1.588	0.946
	192	23.88%	0.153	0.250	0.154	0.248	0.153	0.249	<u>0.201</u>	<u>0.315</u>	0.222	0.334	0.296	0.386	0.386	0.443	0.266	0.368	1.595	0.950
	336	21.02%	0.169	0.268	0.171	0.265	0.169	0.267	<u>0.214</u>	<u>0.329</u>	0.231	0.338	0.300	0.394	0.378	0.443	0.280	0.380	1.617	0.961
	720	17.47%	0.203	0.301	0.210	0.297	0.203	0.301	<u>0.246</u>	<u>0.355</u>	0.254	0.361	0.373	0.439	0.376	0.445	0.283	0.376	1.647	0.975
Exchange	96	45.27%	0.082	0.207	0.089	0.208	0.081	0.203	<u>0.148</u>	<u>0.278</u>	0.197	0.323	0.847	0.752	0.376	1.105	0.968	0.812	0.081	0.196
	192	42.06%	0.167	0.304	0.180	0.300	0.157	0.293	<u>0.271</u>	<u>0.380</u>	0.300	0.369	1.204	0.895	1.748	1.151	1.040	0.851	0.167	0.289
	336	33.69%	0.328	0.432	0.331	0.415	0.305	0.414	<u>0.460</u>	<u>0.500</u>	0.509	0.524	1.672	1.036	1.874	1.172	1.659	1.081	0.305	0.396
	720	46.19%	0.964	0.750	1.033	0.780	0.643	0.601	<u>1.195</u>	<u>0.841</u>	1.447	0.941	2.478	1.310	1.943	1.206	1.941	1.127	0.823	0.681
Traffic	96	30.15%	0.410	0.282	0.410	0.279	0.410	0.282	<u>0.587</u>	<u>0.366</u>	0.613	0.388	0.719	0.391	2.085	0.468	0.684	0.384	2.723	1.079
	192	29.96%	0.423	0.287	0.423	0.284	0.423	0.287	<u>0.604</u>	<u>0.373</u>	0.616	0.382	0.696	0.379	0.867	0.467	0.685	0.390	2.756	1.087
	336	29.95%	0.436	0.295	0.435	0.290	0.436	0.296	<u>0.621</u>	<u>0.383</u>	0.622	<u>0.337</u>	0.777	0.420	0.869	0.469	0.734	0.408	2.791	1.095
	720	25.87%	0.466	0.315	0.464	0.307	0.466	0.315	<u>0.626</u>	<u>0.382</u>	0.660	0.408	0.864	0.472	0.881	0.473	0.717	0.396	2.811	1.097
Weather	96	18.89%	0.176	0.236	0.182	0.232	0.176	0.237	<u>0.217</u>	<u>0.296</u>	0.266	0.336	0.300	0.384	0.896	0.556	0.458	0.490	0.259	0.254
	192	21.01%	0.218	0.276	0.225	0.269	0.220	0.282	<u>0.276</u>	<u>0.336</u>	0.307	0.367	0.598	0.544	0.622	0.624	0.658	0.589	0.309	0.292
	336	22.71%	0.262	0.312	0.271	0.301	0.265	0.319	<u>0.339</u>	<u>0.380</u>	0.359	0.395	0.578	0.523	0.739	0.753	0.797	0.652	0.377	0.338
	720	19.85%	0.326	0.365	0.338	0.348	0.323	0.362	<u>0.403</u>	<u>0.428</u>	0.419	0.428	1.059	0.741	1.004	0.934	0.869	0.675	0.465	0.394
ILI	24	47.86%	1.947	0.985	1.683	0.858	2.215	1.081	<u>3.228</u>	<u>1.260</u>	3.483	1.287	5.764	1.677	1.420	2.012	4.480	1.444	6.587	1.701
	36	36.43%	2.182	1.036	1.703	0.859	1.963	0.963	<u>2.679</u>	<u>1.080</u>	3.103	1.148	4.755	1.467	7.394	2.031	4.799	1.467	7.130	1.884
	48	34.43%	2.256	1.060	1.719	0.884	2.130	1.024	<u>2.622</u>	<u>1.078</u>	2.669	1.085	4.763	1.469	7.551	2.057	4.800	1.468	6.575	1.798
	60	34.33%	2.390	1.104	1.819	0.917	2.368	1.096	<u>2.857</u>	<u>1.157</u>	<u>2.770</u>	<u>1.125</u>	5.264	1.564	7.662	2.100	5.278	1.560	5.893	1.677
ETTh1	96	0.80%	0.375	0.397	0.374	0.394	0.375	0.399	<u>0.376</u>	<u>0.419</u>	0.449	0.459	0.865	0.713	0.664	0.612	0.878	0.740	1.295	0.713
	192	3.57%	0.418	0.429	0.408	0.415	0.405	0.416	<u>0.420</u>	<u>0.448</u>	0.500	0.482	1.008	0.792	0.790	0.681	1.037	0.824	1.325	0.733
	336	6.54%	0.479	0.476	0.429	0.427	0.439	0.443	<u>0.459</u>	<u>0.465</u>	0.521	0.496	1.107	0.809	0.891	0.738	1.238	0.932	1.323	0.744
	720	13.04%	0.624	0.592	0.440	0.453	0.472	0.490	<u>0.506</u>	<u>0.507</u>	0.514	0.512	1.181	0.865	0.963	0.782	1.135	0.852	1.339	0.756
ETTh2	96	19.94%	0.288	0.352	0.277	0.338	0.289	0.353	<u>0.346</u>	<u>0.388</u>	0.358	0.397	3.755	1.525	0.645	0.597	2.116	1.197	0.432	0.422
	192	19.81%	0.377	0.413	0.344	0.381	0.383	0.418	<u>0.429</u>	<u>0.439</u>	0.456	0.452	5.602	1.931	0.788	0.683	4.315	1.635	0.534	0.473
	336	25.93%	0.452	0.461	0.357	0.400	0.448	0.465	<u>0.496</u>	<u>0.487</u>	0.482	0.486	4.721	1.835	0.907	0.747	1.124	1.604	0.591	0.508
	720	14.25%	0.698	0.595	0.394	0.436	0.605	0.551	<u>0.463</u>	<u>0.474</u>	0.515	0.511	3.647	1.625	0.963	0.783	3.188	1.540	0.588	0.517
ETTm1	96	21.10%	0.308	0.352	0.306	0.348	0.299	0.343	<u>0.379</u>	<u>0.419</u>	0.505	0.475	0.672	0.571	0.543	0.510	0.600	0.546	1.214	0.665
	192	21.36%	0.340	0.369	0.349	0.375	0.335	0.365	<u>0.426</u>	<u>0.441</u>	0.553	0.496	0.795	0.669	0.557	0.537	0.837	0.700	1.261	0.690
	336	17.07%	0.376	0.393	0.375	0.388	0.369	0.386	<u>0.445</u>	<u>0.459</u>	0.621	0.537	1.212	0.871	0.754	0.655	1.124	0.832	1.283	0.707
	720	21.73%	0.440	0.435	0.433	0.422	0.425	0.421	<u>0.543</u>	<u>0.490</u>	0.671	0.561	1.166	0.823	0.908	0.724	1.153	0.820	1.319	0.729
ETTm2	96	17.73%	0.168	0.262	0.167	0.255	0.167	0.260	<u>0.203</u>	<u>0.287</u>	0.255	0.339	0.365	0.453	0.435	0.507	0.768	0.642	0.266	0.328
	192	17.84%	0.232	0.308	0.221	0.293	0.224	0.303	<u>0.269</u>	<u>0.328</u>	0.281	0.340	0.533	0.563	0.730	0.673	0.989	0.757	0.340	0.371
	336	15.69%	0.320	0.373	0.274	0.327	0.281	0.342	<u>0.325</u>	<u>0.366</u>	0.339	0.372	1.363	0.887	1.201	0.845	1.334	0.872	0.412	0.410
	720	12.58%	0.413	0.435	0.368	0.384	0.397	0.421	<u>0.421</u>	<u>0.415</u>	0.433	0.432	3.379	1.338	3.625	1.451	3.048	1.328	0.521	0.465

Methods are implemented by us. Other results are from FEDformer [23]

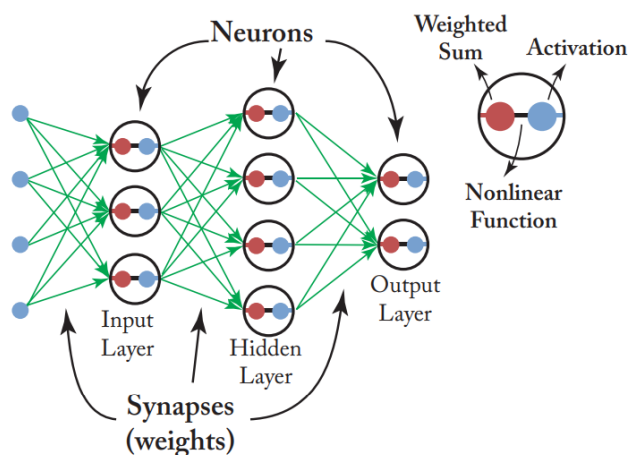
We can see that most of the data, DLinear or NLinear models perform the best compared to all the Transformer based models.

Basics of Neural Networks

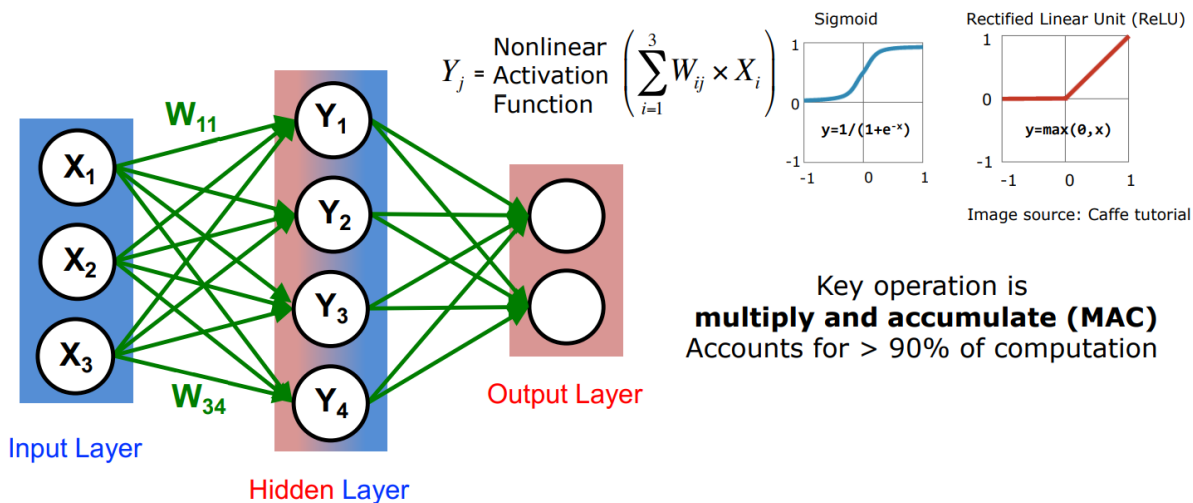
The human brain contains about 86 billion neurons, which are its basic computational units. Each neuron has dendrites that receive input signals, an axon that sends output signals, and synapses that connect it to other neurons. When a neuron receives signals, each synapse scales the input by a certain strength (weight), and the neuron applies a nonlinear function to produce an output signal (activation). The brain has around 10^{14} – 10^{15} synapses, creating an enormous network. Learning in the brain happens by adjusting these synaptic weights—stronger or weaker connections lead to different responses—while the overall structure of the brain remains mostly stable. The weighted sum representation of the neuron is also shown.



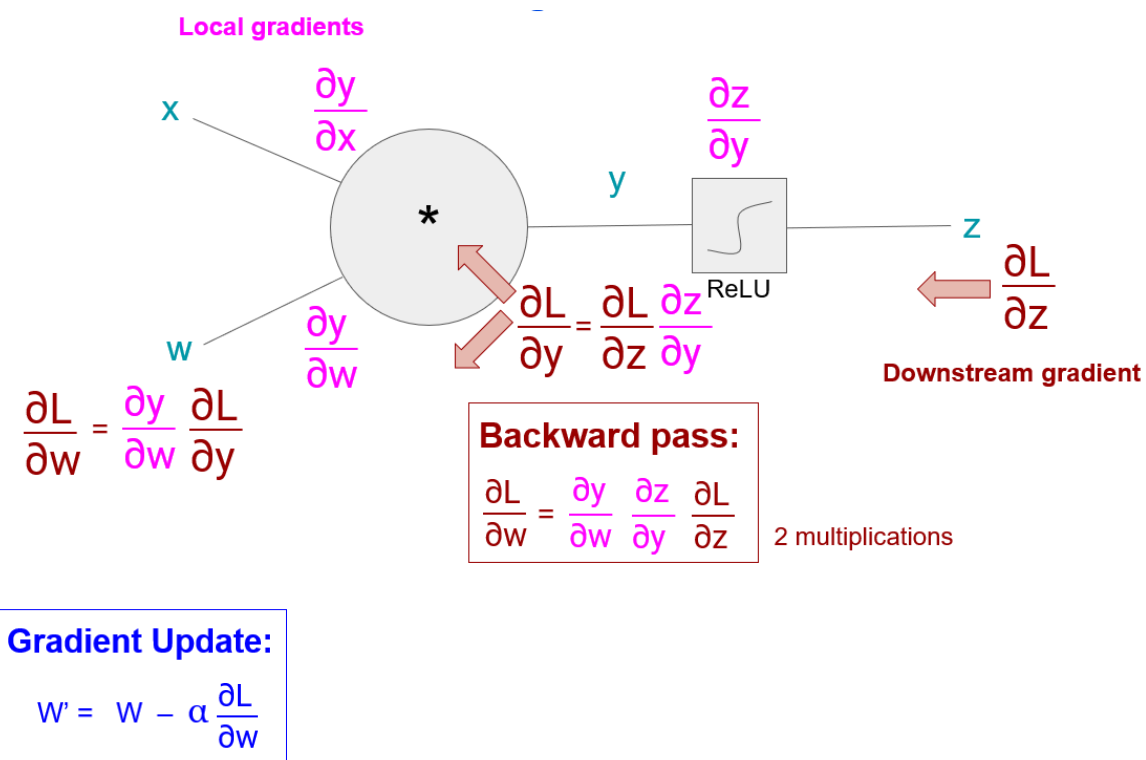
Artificial neural networks are inspired by how biological neurons compute. Each artificial neuron takes input values, multiplies them by weights (similar to synapses), adds them together to form a weighted sum, and then applies a nonlinear function to produce an output called an activation. A neural network is organized into layers: the input layer receives data, hidden layers process the information and pass activations forward (there can be multiple hidden layers), and the output layer produces the final result. In this framework, weights represent synaptic strengths, and activations are the signals passed between neurons.



Much more simplified form is below



Training of neural networks is done using gradient descent method. Since gradients passed from the final stage to first stage the algorithm is called back propagation.



In the single-neuron example, backpropagation computes how the loss $E(w)$ changes with respect to the weight w , and then updates the weight using gradient descent. The key idea is simple: measure how much each parameter contributes to the error and adjust it in the direction that reduces the error.

Backpropagation works by applying the chain rule of calculus layer by layer. Starting from the output layer, we compute how the loss changes with respect to each neuron's output. Then we

propagate this error backward through the network, computing gradients for each weight in each layer.

Neural networks with multiple hidden layers (typically 5 to 1000+ layers) are called deep neural networks. They can learn features from the raw data. Neural networks less than 2 layers are called multilayer perceptron (MLP).

Concept of Attention

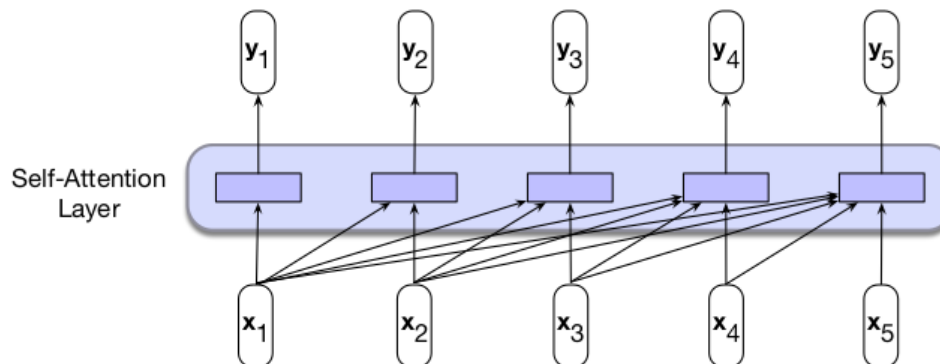
Limitation of Fixed Context Windows: Traditional models use fixed window of previous words & thus can't capture **long-range dependencies** effectively.

Eg: The **chicken** didn't cross the road because **it** was too tired.

(Here word **it** depends on word **chicken** beyond 3 words in prev. example)

Attention Mechanism

- Instead of small **fixed window**, attention **looks at large number of words & assigns importance** dynamically.
- Helps model **long-term dependencies** better.



Key Idea: Compare each input with others to determine relevance in context.

Step 1: Compute scores by comparing each input with all others using **dot product**: $\text{score}(x_i, x_j) = x_i \cdot x_j$ (**Higher score** $\rightarrow$ **More relevance**, dot product gives us the cosine distance)

Step 2: Normalize with softmax: $\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j)) \forall j \leq i$ i.e. convert the weights to probabilities adding to 1.

Step 3: Compute output as weighted sum of inputs:

$$y_i = \sum_{j \leq i} \alpha_{ij} x_j$$

More sophisticated using multiple weight matrices

Query is current input; **Key** is previous input compared to query.; **Value** used to compute output.

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i; \quad \mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i; \quad \mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

$\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$: Weight matrices

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{q}_i \cdot \mathbf{k}_j$$

$$\mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Weight matrices learn specific aspect of language. In multi-head attention we can have multiple attention matrices capturing different aspects of language. Output from multiple heads can be converted to single output using projection matrices.

Transformer Networks

Self-Attention: Key part that captures relationships between words

Residual Connection: Helps retain original input, making training stable

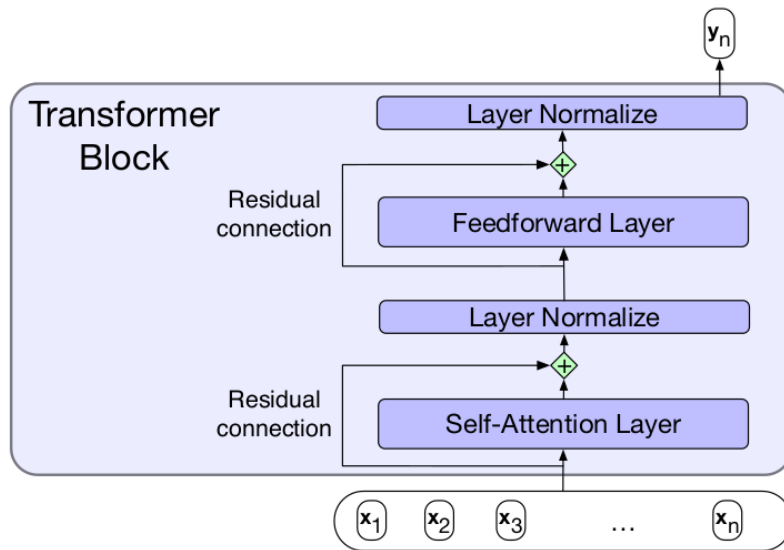
Layer Normalization: Normalizes activations for stable learning and faster convergence.

$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttn}(\mathbf{x}))$$

$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFNN}(\mathbf{z}))$$

$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i \quad \sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2} \quad \hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm} = \gamma \hat{\mathbf{x}} + \beta$$

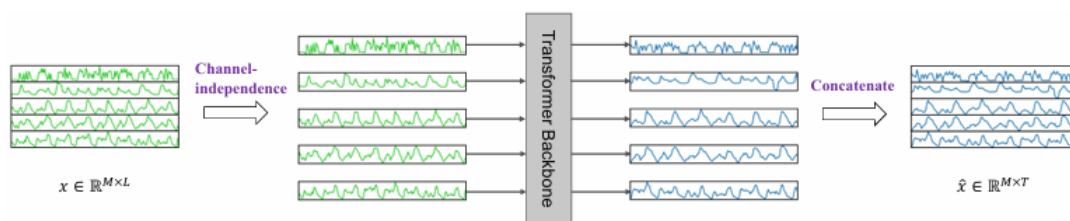


PatchTST (Patch Time Series Transformer) (published in 2023 and 4600 citations)

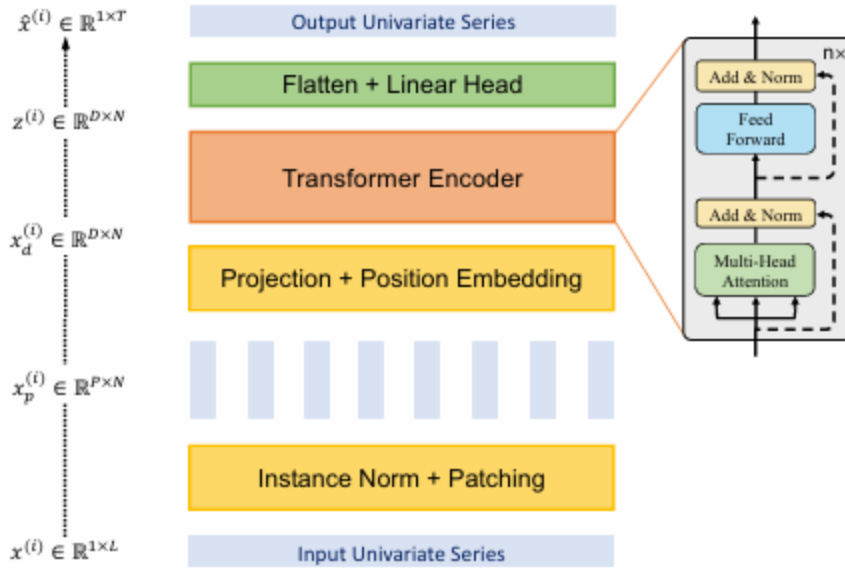
To resolve the problems of transformer models overfitting or being complex for time series data, this uses the concept of patching.

Patching: Each univariate time series is first divided into smaller segments called patches, which may either overlap or remain separate depending on the stride. If the patch length is P and the stride is S , the original sequence of length L is converted into approximately $(L-P)/S+2$ patches. This process effectively reduces the number of input tokens from L to roughly L/S . As a result, both memory usage and the computational cost of attention—which grows quadratically with sequence length—are significantly reduced. This reduction allows the model to process much longer historical sequences within practical training time and GPU memory limits, thereby improving its ability to capture long-term dependencies and enhancing forecasting performance.

All channels (multiple series) are processed independently in this model



The PatchTST backbone is shown below



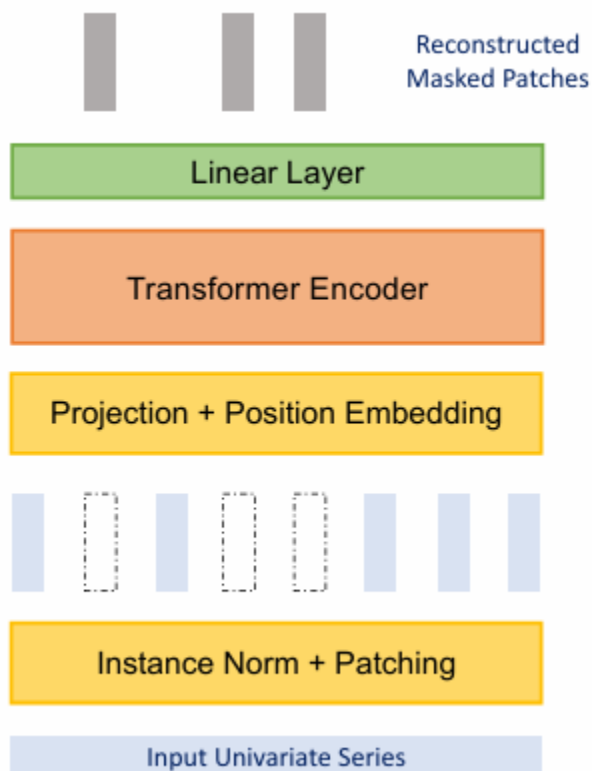
P= patch length, N = no of patches per series

Trained using the MSE loss.

Models	PatchTST/64		PatchTST/42		DLinear		FEDformer		Autoformer		Informer		Pyraformer		LogTrans		
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
Weather	96	0.149	0.198	<u>0.152</u>	<u>0.199</u>	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405	0.896	0.556	0.458	0.490
	192	0.194	0.241	<u>0.197</u>	<u>0.243</u>	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434	0.622	0.624	0.658	0.589
	336	0.245	0.282	<u>0.249</u>	<u>0.283</u>	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543	0.739	0.753	0.797	0.652
	720	0.314	0.334	<u>0.320</u>	<u>0.335</u>	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705	1.004	0.934	0.869	0.675
Traffic	96	0.360	0.249	<u>0.367</u>	<u>0.251</u>	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410	2.085	0.468	0.684	0.384
	192	0.379	0.256	<u>0.385</u>	<u>0.259</u>	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435	0.867	0.467	0.685	0.390
	336	0.392	0.264	<u>0.398</u>	<u>0.265</u>	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434	0.869	0.469	0.734	0.408
	720	0.432	0.286	<u>0.434</u>	<u>0.287</u>	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466	0.881	0.473	0.717	0.396
Electricity	96	0.129	0.222	<u>0.130</u>	0.222	0.140	0.237	0.186	0.302	0.196	0.313	0.304	0.393	0.386	0.449	0.258	0.357
	192	0.147	0.240	<u>0.148</u>	0.240	0.153	0.249	0.197	0.311	0.211	0.324	0.327	0.417	0.386	0.443	0.266	0.368
	336	0.163	0.259	<u>0.167</u>	<u>0.261</u>	0.169	0.267	0.213	0.328	0.214	0.327	0.333	0.422	0.378	0.443	0.280	0.380
	720	0.197	0.290	<u>0.202</u>	<u>0.291</u>	0.203	0.301	0.233	0.344	0.236	0.342	0.351	0.427	0.376	0.445	0.283	0.376
ILI	24	1.319	0.754	<u>1.522</u>	<u>0.814</u>	2.215	1.081	2.624	1.095	2.906	1.182	4.657	1.449	1.420	2.012	4.480	1.444
	36	<u>1.579</u>	<u>0.870</u>	1.430	0.834	1.963	0.963	2.516	1.021	2.585	1.038	4.650	1.463	7.394	2.031	4.799	1.467
	48	1.553	0.815	<u>1.673</u>	<u>0.854</u>	2.130	1.024	2.505	1.041	3.024	1.145	5.004	1.542	7.551	2.057	4.800	1.468
	60	1.470	0.788	<u>1.529</u>	<u>0.862</u>	2.368	1.096	2.742	1.122	2.761	1.114	5.071	1.543	7.662	2.100	5.278	1.560
ETTh1	96	0.370	<u>0.400</u>	<u>0.375</u>	0.399	<u>0.375</u>	0.399	<u>0.376</u>	<u>0.415</u>	<u>0.435</u>	<u>0.446</u>	<u>0.941</u>	<u>0.769</u>	<u>0.664</u>	<u>0.612</u>	<u>0.878</u>	<u>0.740</u>
	192	<u>0.413</u>	<u>0.429</u>	<u>0.414</u>	<u>0.421</u>	0.405	0.416	<u>0.423</u>	<u>0.446</u>	<u>0.456</u>	<u>0.457</u>	<u>1.007</u>	<u>0.786</u>	<u>0.790</u>	<u>0.681</u>	<u>1.037</u>	<u>0.824</u>
	336	0.422	<u>0.440</u>	<u>0.431</u>	0.436	<u>0.439</u>	<u>0.443</u>	<u>0.444</u>	<u>0.462</u>	<u>0.486</u>	<u>0.487</u>	<u>1.038</u>	<u>0.784</u>	<u>0.891</u>	<u>0.738</u>	<u>1.238</u>	<u>0.932</u>
	720	0.447	<u>0.468</u>	<u>0.449</u>	0.466	<u>0.472</u>	<u>0.490</u>	<u>0.469</u>	<u>0.492</u>	<u>0.515</u>	<u>0.517</u>	<u>1.144</u>	<u>0.857</u>	<u>0.963</u>	<u>0.782</u>	<u>1.135</u>	<u>0.852</u>
ETTh2	96	0.274	<u>0.337</u>	0.274	0.336	<u>0.289</u>	<u>0.353</u>	<u>0.332</u>	<u>0.374</u>	<u>0.332</u>	<u>0.368</u>	<u>1.549</u>	<u>0.952</u>	<u>0.645</u>	<u>0.597</u>	<u>2.116</u>	<u>1.197</u>
	192	<u>0.341</u>	<u>0.382</u>	0.339	0.379	<u>0.383</u>	<u>0.418</u>	<u>0.407</u>	<u>0.446</u>	<u>0.426</u>	<u>0.434</u>	<u>3.792</u>	<u>1.542</u>	<u>0.788</u>	<u>0.683</u>	<u>4.315</u>	<u>1.635</u>
	336	0.329	<u>0.384</u>	<u>0.331</u>	0.380	<u>0.448</u>	<u>0.465</u>	<u>0.400</u>	<u>0.447</u>	<u>0.477</u>	<u>0.479</u>	<u>4.215</u>	<u>1.642</u>	<u>0.907</u>	<u>0.747</u>	<u>1.124</u>	<u>1.604</u>
	720	0.379	0.422	0.379	0.422	<u>0.605</u>	<u>0.551</u>	<u>0.412</u>	<u>0.469</u>	<u>0.453</u>	<u>0.490</u>	<u>3.656</u>	<u>1.619</u>	<u>0.963</u>	<u>0.783</u>	<u>3.188</u>	<u>1.540</u>
ETTm1	96	<u>0.293</u>	<u>0.346</u>	0.290	0.342	<u>0.299</u>	<u>0.343</u>	<u>0.326</u>	<u>0.390</u>	<u>0.510</u>	<u>0.492</u>	<u>0.626</u>	<u>0.560</u>	<u>0.543</u>	<u>0.510</u>	<u>0.600</u>	<u>0.546</u>
	192	<u>0.333</u>	<u>0.370</u>	0.332	<u>0.369</u>	<u>0.335</u>	0.365	<u>0.365</u>	<u>0.415</u>	<u>0.514</u>	<u>0.495</u>	<u>0.725</u>	<u>0.619</u>	<u>0.557</u>	<u>0.537</u>	<u>0.837</u>	<u>0.700</u>
	336	<u>0.369</u>	<u>0.392</u>	0.366	<u>0.392</u>	<u>0.369</u>	0.386	<u>0.392</u>	<u>0.425</u>	<u>0.510</u>	<u>0.492</u>	<u>1.005</u>	<u>0.741</u>	<u>0.754</u>	<u>0.655</u>	<u>1.124</u>	<u>0.832</u>
	720	0.416	0.420	<u>0.420</u>	<u>0.424</u>	<u>0.425</u>	<u>0.421</u>	<u>0.446</u>	<u>0.458</u>	<u>0.527</u>	<u>0.493</u>	<u>1.133</u>	<u>0.845</u>	<u>0.908</u>	<u>0.724</u>	<u>1.153</u>	<u>0.820</u>
ETTm2	96	<u>0.166</u>	<u>0.256</u>	0.165	0.255	<u>0.167</u>	<u>0.260</u>	<u>0.180</u>	<u>0.271</u>	<u>0.205</u>	<u>0.293</u>	<u>0.355</u>	<u>0.462</u>	<u>0.435</u>	<u>0.507</u>	<u>0.768</u>	<u>0.642</u>
	192	<u>0.223</u>	<u>0.296</u>	0.220	0.292	<u>0.224</u>	<u>0.303</u>	<u>0.252</u>	<u>0.318</u>	<u>0.278</u>	<u>0.336</u>	<u>0.595</u>	<u>0.586</u>	<u>0.730</u>	<u>0.673</u>	<u>0.989</u>	<u>0.757</u>
	336	0.274	0.329	<u>0.278</u>	0.329	<u>0.281</u>	<u>0.342</u>	<u>0.324</u>	<u>0.364</u>	<u>0.343</u>	<u>0.379</u>	<u>1.270</u>	<u>0.871</u>	<u>1.201</u>	<u>0.845</u>	<u>1.334</u>	<u>0.872</u>
	720	0.362	0.385	<u>0.367</u>	0.385	<u>0.397</u>	<u>0.421</u>	<u>0.410</u>	<u>0.420</u>	<u>0.414</u>	<u>0.419</u>	<u>3.001</u>	<u>1.267</u>	<u>3.625</u>	<u>1.451</u>	<u>3.048</u>	<u>1.328</u>

We can see that PatchTST performs better than DLinear and other transformer models.

Concept of self-supervised training for representation learning



Mask some of the input patches and try to predict the masked patches.

This can be used for transfer learning i.e., trained on one dataset can be used as base model to another problem with different dataset. By using small amount of new dataset we can get superior performance than using the supervised learning.

Models		PatchTST						DLinear		FEDformer		Autoformer		Informer	
		Fine-tuning		Lin. Prob.		Sup.									
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	0.145	0.195	0.163	0.216	0.152	0.199	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405
	192	0.193	0.243	0.205	0.252	0.197	0.243	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434
	336	0.244	0.280	0.253	0.289	0.249	0.283	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543
	720	0.321	0.337	0.320	0.336	0.320	0.335	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705
Traffic	96	0.388	0.273	0.400	0.288	0.367	0.251	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410
	192	0.400	0.277	0.412	0.293	0.385	0.259	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435
	336	0.408	0.280	0.425	0.307	0.398	0.265	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434
	720	0.447	0.310	0.457	0.317	0.434	0.287	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466

Table 5: Transfer learning task: PatchTST is pre-trained on Electricity dataset and the model is transferred to other datasets. The best results are in **bold** and the second best are underlined.

Finetuning: Training after representation learning completed and using those weights.

Linear Probing: Only one top layer is finetuned instead of all the model

Supervised Training: Trained on its own dataset without any pretraining

TS Mixer (Time Series Mixer) (704 citations and published in 2023 by google research)

Motivated by the DLinear and NLinear model, they used MLP networks instead of weighted sum to improve the performance. Also they used architectural mixing of the multivariate series to improve the performance.

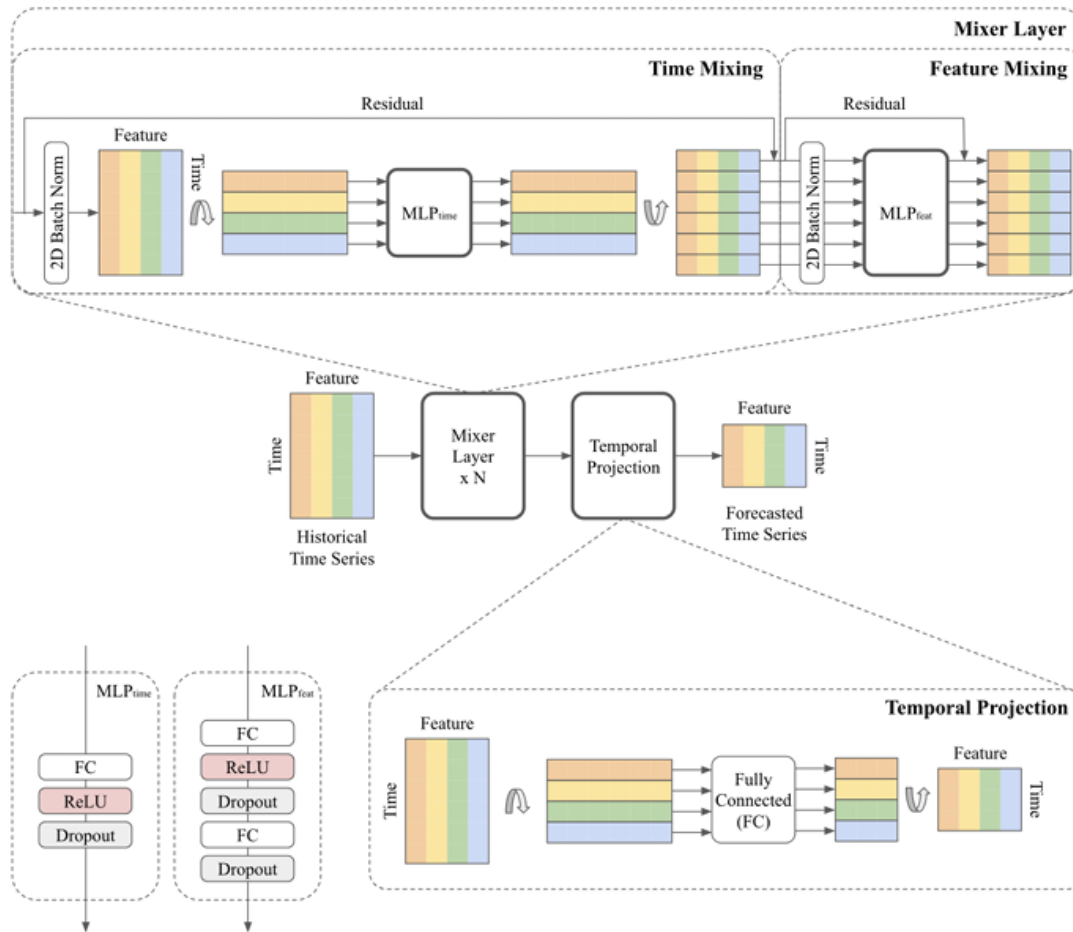


Figure 1: TSMixer for multivariate time series forecasting. The columns of the inputs means different features/variates and the rows are time steps. The fully-connected operations are row-wise. TSMixer contains interleaving time-mixing and feature-mixing MLPs to aggregate information. The number of mixer layer is denoted as N . The time-mixing MLPs are shared across all features and the feature-mixing MLPs are shared across all of the time steps. The design allow TSMixer to automatically adapt the use of both temporal and cross-variant information with limited number of parameters for superior generalization. The extension with auxiliary information is also explored in this paper.

Note that it uses the concept of normalization and the residual connections from the transformer model.

Description of components

2D Batch Norm: Normalization (mean, variance normalization) is done across batch of samples and across all the time series (time and as well as the series dimension).

Temporal Projection Given an input matrix $\mathbf{X} \in \mathbb{R}^{L \times C}$, the Temporal Projection (TP) is a linear layer that acts on the columns of $\mathbf{X}$ (denoted as $\mathbf{X}_{*,i}$) and is shared across all columns to project the time series from the input length to the prediction length. The operation is defined as:

$$\text{TP}_{L \rightarrow T}(\mathbf{X})_{*,i} = \mathbf{W}_1 \mathbf{X}_{*,i} + \mathbf{b}_1, \forall i = 1, \dots, C, \quad (4)$$

where $\mathbf{W}_1 \in \mathbb{R}^{L \times T}$ and $\mathbf{b}_1 \in \mathbb{R}^T$ are the weights and biases of the linear layer respectively. The subscript $L \rightarrow T$ denotes the mapping between input and output dimensions.

Time Mixing Similar to the Temporal Projection, the Time Mixing (TM) acts on all columns of $\mathbf{X}$ and applies commonly used deep learning layers to perform temporal feature transformation. The operation is defined as:

$$\begin{aligned} \text{TM}(\mathbf{X})_{*,i} = & \\ & \text{Norm} \left(\mathbf{X}_{*,i} + \text{Drop} \left(\sigma \left(\text{TP}_{L \rightarrow L}(\mathbf{X})_{*,i} \right) \right) \right), \end{aligned} \quad \forall i = 1, \dots, C, \quad (5)$$

where $\sigma(\cdot)$ is an activation function, $\text{Drop}(\cdot)$ is dropout and $\text{Norm}(\cdot)$ can be layer normalization or batch normalization. It is important to note that the normalization is applied on the entire matrix (along both time and feature domain), rather than row-by-row (along the feature domain) as in Transformer-based models. The TM block allows TSMixer to effectively capture temporal dependencies in the time series data.

Time-mixing is actually an MLP. Dropout in the above is used reduce overfitting by randomly dropping some weights in the network.

Feature Mixing The Feature Mixing (FM) is a two-layer residual MLP that acts on the rows of the input matrix $\mathbf{X} \in \mathbb{R}^{L \times C}$ and is shared across all rows. The block is designed to model feature transformations and is applied to each row $\mathbf{X}_{j,*}$ of the input matrix. The operation is defined as:

$$\begin{aligned} \mathbf{U}_{j,*} &= \text{Drop}(\sigma(\mathbf{W}_2 \mathbf{X}_{j,*} + \mathbf{b}_2)), \\ \text{FM}_{C \rightarrow C}(\mathbf{X})_{j,*} &= \text{Norm}(\mathbf{X}_{j,*} + \text{Drop}(\mathbf{W}_3 \mathbf{U}_{j,*} + \mathbf{b}_3)), \\ &\quad \forall j = 1, \dots, L, \end{aligned}$$

where $\mathbf{W}_2, \mathbf{W}_3 \in \mathbb{R}^{C \times C}$ and $\mathbf{b}_2, \mathbf{b}_3 \in \mathbb{R}^C$.

Table 3: Evaluation results on the long-term forecasting datasets. The numbers of models marked with “*” are obtained from [Nie et al. \(2023\)](#). The best numbers in each row are shown in **bold** and the second best numbers are underlined. We skip TMix-Only in comparisons as it performs similar to TSMixer. The last row shows the average percentage of MSE improvement of TSMixer over other methods.

		Multivariate Model								Univariate Model							
Models		TSMixer		TFT	FEDformer*		Autoformer*		Informer*		TMix-Only		Linear		PatchTST*		
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.361	0.392	0.674	0.634	0.376	0.415	0.435	0.446	0.941	0.769	0.359	0.391	0.368	0.392	0.370	0.400
	192	0.404	0.418	0.858	0.704	0.423	0.446	0.456	0.457	1.007	0.786	0.402	0.415	0.404	0.415	0.413	0.429
	336	0.420	0.431	0.900	0.731	0.444	0.462	0.486	0.487	1.038	0.784	0.420	0.434	0.436	0.439	0.422	0.440
	720	0.463	0.472	0.745	0.666	0.469	0.492	0.515	0.517	1.144	0.857	0.453	0.467	0.481	0.495	0.447	0.468
ETTTh2	96	0.274	0.341	0.409	0.505	0.332	0.374	0.332	0.368	1.549	0.952	0.275	0.342	0.297	0.363	0.274	0.337
	192	0.339	0.385	0.953	0.651	0.407	0.446	0.426	0.434	3.792	1.542	0.339	0.386	0.398	0.429	0.341	0.382
	336	0.361	0.406	1.006	0.709	0.400	0.447	0.477	0.479	4.215	1.642	0.366	0.413	0.500	0.491	0.329	0.384
	720	0.445	0.470	1.187	0.816	0.412	0.469	0.453	0.490	3.656	1.619	0.437	0.465	0.795	0.633	0.379	0.422
ETTh1	96	0.285	0.339	0.752	0.626	0.326	0.390	0.510	0.492	0.626	0.560	0.284	0.338	0.303	0.346	0.293	0.346
	192	0.327	0.365	0.752	0.649	0.365	0.415	0.514	0.495	0.725	0.619	0.324	0.362	0.335	0.365	0.333	0.370
	336	0.356	0.382	0.810	0.674	0.392	0.425	0.510	0.492	1.005	0.741	0.359	0.384	0.365	0.384	0.369	0.392
	720	0.419	0.414	0.849	0.695	0.446	0.458	0.527	0.493	1.133	0.845	0.419	0.414	0.419	0.415	0.416	0.420
ETTTh2	96	0.163	0.252	0.386	0.472	0.180	0.271	0.205	0.293	0.355	0.462	0.162	0.249	0.170	0.266	0.166	0.256
	192	0.216	0.290	0.739	0.626	0.252	0.318	0.278	0.336	0.595	0.586	0.220	0.293	0.236	0.317	0.223	0.296
	336	0.268	0.324	0.477	0.494	0.324	0.364	0.343	0.379	1.270	0.871	0.269	0.326	0.308	0.369	0.274	0.329
	720	0.420	0.422	0.523	0.537	0.410	0.420	0.414	0.419	3.001	1.267	0.358	0.382	0.435	0.449	0.362	0.385
Weather	96	0.145	0.198	0.441	0.474	0.238	0.314	0.249	0.329	0.354	0.405	0.145	0.196	0.170	0.229	0.149	0.198
	192	0.191	0.242	0.699	0.599	0.275	0.329	0.325	0.370	0.419	0.434	0.190	0.240	0.213	0.268	0.194	0.241
	336	0.242	0.280	0.693	0.596	0.339	0.377	0.351	0.391	0.583	0.543	0.240	0.279	0.257	0.305	0.245	0.282
	720	0.320	0.336	1.038	0.753	0.389	0.409	0.415	0.426	0.916	0.705	0.325	0.339	0.318	0.356	0.314	0.334
Electricity	96	0.131	0.229	0.295	0.376	0.186	0.302	0.196	0.313	0.304	0.393	0.132	0.225	0.135	0.232	0.129	0.222
	192	0.151	0.246	0.327	0.397	0.197	0.311	0.211	0.324	0.327	0.417	0.152	0.243	0.149	0.246	0.147	0.240
	336	0.161	0.261	0.298	0.380	0.213	0.328	0.214	0.327	0.333	0.422	0.166	0.260	0.164	0.263	0.163	0.259
	720	0.197	0.293	0.338	0.412	0.233	0.344	0.236	0.342	0.351	0.427	0.200	0.291	0.199	0.297	0.197	0.290
Traffic	96	0.376	0.264	0.678	0.362	0.576	0.359	0.597	0.371	0.733	0.410	0.370	0.258	0.395	0.274	0.360	0.249
	192	0.397	0.277	0.664	0.355	0.610	0.380	0.607	0.382	0.777	0.435	0.390	0.268	0.406	0.279	0.379	0.256
	336	0.413	0.290	0.679	0.354	0.608	0.375	0.623	0.387	0.776	0.434	0.404	0.276	0.416	0.286	0.392	0.264
	720	0.444	0.306	0.610	0.326	0.621	0.375	0.639	0.395	0.827	0.466	0.443	0.297	0.454	0.308	0.432	0.286
TSMixer MSE Imp.				51.94%		16.69%		24.51%		62.40%		-0.66%		6.77%		-1.53%	

TS Mizer performs better than the PatchTST

TimeMixer (Published in 2024, 1068 citations)

Time series behave differently at different time scales. For example, hourly data shows detailed daily patterns, while daily data highlights trends like holidays, and yearly data captures long-term trends such as economic changes. Because of this, it is useful to analyze data at multiple scales to separate short-term and long-term patterns. In time series forecasting, future values are influenced by patterns at multiple time scales rather than a single one. To address this, the model is designed to combine information across different scales, allowing it to capture both distinct patterns at each level and the complementary relationships between them for improved predictions.

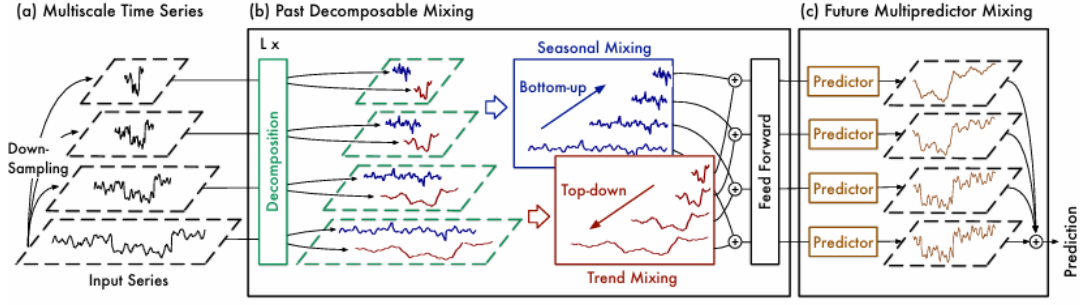


Figure 1: Overall architecture of TimeMixer, which consists of Past-Decomposable Mixing and Future-Multipredictor-Mixing for past observations and future predictions respectively.

Downsampling: Downsample the past observations into M scales by average pooling (represent multiple samples as average of those samples) and finally obtain a set of multiscale time series.

Past Decomposable Mixing:

- (i) **Decomposition:** Decompose each series into trend and seasonality using a pretrained model. We can also use trend and seasonality decomposition of DLinear here instead.
- (ii) **Seasonal Mixing:** In seasonal mixing, the model follows a bottom-up approach, where detailed patterns from fine (short-term) time scales are passed upward to coarser (long-term) scales. This helps the model use fine-level details to improve how it captures seasonal patterns at higher levels. This is achieved using a two linear layers with an activation function.
- (iii) **Trend Mixing:** Higher (coarser) time scales capture clearer long-term trends compared to finer scales. So, the model uses a **top-down approach**, where this high-level information is passed down to guide and improve trend modeling at finer time scales. This is achieved using a two linear layers with an activation function.

The equations are

$$\mathbf{s}_m^l, \mathbf{t}_m^l = \text{SeriesDecomp}(\mathbf{x}_m^l), m \in \{0, \dots, M\},$$

$$\mathcal{X}^l = \mathcal{X}^{l-1} + \text{FeedForward} \left(\text{S-Mix}(\{\mathbf{s}_m^l\}_{m=0}^M) + \text{T-Mix}(\{\mathbf{t}_m^l\}_{m=0}^M) \right),$$

Note this process is repeated multiple times similar to deep learning to get all the scales enhanced using the information from other scales.

Future Multipredictor Mixing

It consists of single layer network for predictor and sums all the multiscale series to predict the future samples.

Table 2: Long-term forecasting results. All the results are averaged from 4 different prediction lengths, that is $\{96, 192, 336, 720\}$. A lower MSE or MAE indicates a better prediction. We fix the input length as 96 for all experiments. See Table [I3](#) in Appendix for the full results.

Models	TimeMixer (Ours)	PatchTST (2023)	TimesNet (2023a)	Crossformer (2023)	MICN (2023)	FiLM (2022a)	DLinear (2023)	FEDformer (2022b)	Stationary (2022b)	Autoformer (2021)	Informer (2021)
Metric	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE
Weather	0.240 0.271	0.265 0.285	0.251 0.294	0.264 0.320	0.268 0.321	0.271 0.291	0.265 0.315	0.309 0.360	0.288 0.314	0.338 0.382	0.634 0.548
Solar-Energy	0.216 0.280	0.287 0.333	0.403 0.374	0.406 0.442	0.283 0.358	0.380 0.371	0.330 0.401	0.328 0.383	0.350 0.390	0.586 0.557	0.331 0.381
Electricity	0.182 0.272	0.216 0.318	0.193 0.304	0.244 0.334	0.196 0.309	0.223 0.302	0.225 0.319	0.214 0.327	0.193 0.296	0.227 0.338	0.311 0.397
Traffic	0.484 0.297	0.529 0.341	0.620 0.336	0.667 0.426	0.593 0.356	0.637 0.384	0.625 0.383	0.610 0.376	0.624 0.340	0.628 0.379	0.764 0.416
ETTh1	0.447 0.440	0.516 0.484	0.495 0.450	0.529 0.522	0.475 0.480	0.516 0.483	0.461 0.457	0.498 0.484	0.570 0.537	0.496 0.487	1.040 0.795
ETTh2	0.364 0.395	0.391 0.411	0.414 0.427	0.942 0.684	0.574 0.531	0.402 0.420	0.563 0.519	0.437 0.449	0.526 0.516	0.450 0.459	4.431 1.729
ETTm1	0.381 0.395	0.406 0.407	0.400 0.406	0.513 0.495	0.423 0.422	0.411 0.402	0.404 0.408	0.448 0.452	0.481 0.456	0.588 0.517	0.961 0.734
ETTm2	0.275 0.323	0.290 0.334	0.291 0.333	0.757 0.610	0.353 0.402	0.287 0.329	0.354 0.402	0.305 0.349	0.306 0.347	0.327 0.371	1.410 0.810

Table 23: Unified and searched hyperparameter results for additional baselines in the long-term forecasting task. We compare extensive competitive models under different prediction lengths. Avg is averaged from all four prediction lengths, that is 96, 192, 336, 720. All these baselines are reproduced by their official code. For the searched hyperparameter setting, we follow the searching strategy described in Appendix E. Especially for TSMixer, we reproduced it in Pytorch (Paszke et al., 2019).

Models		Unified Hyperparameter								Searched Hyperparameter							
		TimeMixer (Ours)		Scaleformer (2023)		MTSMixer (2023)		TSMixer (2023)		TimeMixer (Ours)		Scaleformer (2023)		MTSMixer (2023)		TSMixer (2023)	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	0.163	0.209	0.220	0.289	0.173	0.224	0.175	0.247	0.147	0.197	0.192	0.241	0.167	0.221	0.149	0.198
	192	0.208	0.250	0.341	0.385	0.219	0.261	0.224	0.294	0.189	0.239	0.220	0.288	0.208	0.250	0.201	0.251
	336	0.251	0.287	0.463	0.455	0.274	0.300	0.262	0.326	0.241	0.280	0.288	0.324	0.298	0.302	0.287	0.291
	720	0.339	0.341	0.640	0.565	0.365	0.359	0.349	0.348	0.310	0.330	0.321	0.360	0.344	0.339	0.320	0.336
	Avg	0.240	0.271	0.416	0.423	0.258	0.286	0.253	0.304	0.222	0.262	0.248	0.304	0.254	0.278	0.240	0.269
Solar-Energy	96	0.189	0.259	0.271	0.331	0.217	0.272	0.216	0.294	0.167	0.220	0.224	0.328	0.199	0.251	0.190	0.272
	192	0.222	0.283	0.288	0.332	0.258	0.299	0.294	0.359	0.187	0.249	0.247	0.312	0.221	0.275	0.207	0.278
	336	0.231	0.292	0.358	0.412	0.278	0.310	0.302	0.367	0.200	0.258	0.274	0.308	0.231	0.281	0.246	0.304
	720	0.223	0.285	0.377	0.437	0.293	0.321	0.311	0.372	0.215	0.250	0.335	0.384	0.270	0.301	0.274	0.308
	Avg	0.216	0.280	0.323	0.378	0.261	0.300	0.280	0.348	0.192	0.244	0.270	0.333	0.230	0.277	0.229	0.283
Electricity	96	0.153	0.247	0.182	0.297	0.173	0.270	0.190	0.299	0.129	0.224	0.162	0.274	0.154	0.267	0.142	0.234
	192	0.166	0.256	0.188	0.300	0.186	0.280	0.216	0.323	0.140	0.220	0.171	0.284	0.168	0.272	0.154	0.248
	336	0.185	0.277	0.210	0.324	0.204	0.297	0.226	0.334	0.161	0.255	0.192	0.304	0.182	0.281	0.161	0.262
	720	0.225	0.310	0.232	0.339	0.241	0.326	0.250	0.353	0.194	0.287	0.238	0.332	0.212	0.321	0.209	0.304
	Avg	0.182	0.272	0.203	0.315	0.201	0.293	0.220	0.327	0.156	0.246	0.191	0.298	0.179	0.286	0.167	0.262
Traffic	96	0.462	0.285	0.564	0.351	0.523	0.357	0.499	0.344	0.360	0.249	0.409	0.281	0.514	0.338	0.398	0.272
	192	0.473	0.296	0.570	0.349	0.535	0.367	0.540	0.370	0.375	0.250	0.418	0.294	0.519	0.351	0.402	0.281
	336	0.498	0.296	0.576	0.349	0.566	0.379	0.557	0.378	0.385	0.270	0.427	0.294	0.557	0.361	0.412	0.294
	720	0.506	0.313	0.602	0.360	0.608	0.397	0.586	0.397	0.430	0.281	0.518	0.356	0.569	0.362	0.448	0.311
	Avg	0.484	0.297	0.578	0.352	0.558	0.375	0.546	0.372	0.387	0.262	0.443	0.307	0.539	0.354	0.415	0.290
ETTh1	96	0.375	0.400	0.401	0.428	0.418	0.437	0.387	0.411	0.361	0.390	0.381	0.412	0.397	0.428	0.370	0.402
	192	0.429	0.421	0.471	0.478	0.463	0.460	0.441	0.437	0.409	0.414	0.445	0.441	0.452	0.466	0.406	0.414
	336	0.484	0.458	0.527	0.498	0.516	0.478	0.507	0.467	0.430	0.429	0.501	0.484	0.487	0.462	0.424	0.434
	720	0.498	0.482	0.578	0.547	0.532	0.549	0.527	0.548	0.445	0.460	0.544	0.528	0.510	0.506	0.471	0.479
	Avg	0.447	0.440	0.495	0.488	0.482	0.481	0.466	0.467	0.411	0.423	0.468	0.466	0.461	0.464	0.418	0.432
ETTh2	96	0.289	0.341	0.368	0.398	0.343	0.378	0.308	0.357	0.271	0.330	0.340	0.394	0.328	0.367	0.271	0.339
	192	0.372	0.392	0.431	0.446	0.422	0.425	0.395	0.404	0.317	0.402	0.401	0.414	0.404	0.426	0.344	0.397
	336	0.386	0.414	0.486	0.474	0.462	0.460	0.428	0.434	0.332	0.396	0.437	0.448	0.406	0.434	0.360	0.400
	720	0.412	0.434	0.517	0.522	0.476	0.475	0.443	0.451	0.342	0.408	0.469	0.471	0.448	0.463	0.428	0.461
	Avg	0.364	0.395	0.451	0.460	0.426	0.435	0.394	0.412	0.316	0.384	0.412	0.432	0.397	0.422	0.350	0.399
ETTm1	96	0.320	0.357	0.383	0.408	0.344	0.378	0.331	0.378	0.291	0.340	0.338	0.375	0.316	0.362	0.288	0.336
	192	0.361	0.381	0.417	0.421	0.397	0.408	0.386	0.399	0.327	0.365	0.392	0.406	0.374	0.391	0.332	0.374
	336	0.390	0.404	0.437	0.448	0.429	0.430	0.426	0.421	0.360	0.381	0.410	0.426	0.408	0.411	0.358	0.381
	720	0.454	0.441	0.512	0.481	0.489	0.460	0.489	0.465	0.415	0.417	0.481	0.476	0.472	0.454	0.420	0.417
	Avg	0.381	0.395	0.438	0.440	0.415	0.419	0.408	0.416	0.348	0.375	0.406	0.421	0.393	0.405	0.350	0.377
ETTm2	96	0.175	0.258	0.201	0.297	0.191	0.278	0.179	0.282	0.164	0.254	0.192	0.274	0.187	0.268	0.160	0.249
	192	0.237	0.299	0.261	0.324	0.258	0.320	0.244	0.305	0.223	0.295	0.248	0.322	0.237	0.301	0.228	0.299
	336	0.298	0.340	0.328	0.366	0.319	0.357	0.320	0.357	0.279	0.330	0.301	0.348	0.299	0.352	0.269	0.328
	720	0.391	0.396	0.424	0.417	0.417	0.412	0.419	0.432	0.359	0.383	0.411	0.398	0.413	0.419	0.421	0.426
	Avg	0.275	0.323	0.303	0.351	0.296	0.342	0.290	0.344	0.256	0.315	0.288	0.336	0.284	0.335	0.270	0.326

It performs better all the previous models including DLinear, PatchTST, TS Mixer and all the previous transformer models.

FreEFormer (Frequency Enhanced Transformer) (Published in 2025 and has got 33 citations)

Discrete Fourier Transform: We can express the time series as a sum of sinusoidal signal with different frequency. DFT decomposes the time series signal into multiple sinusoidal signals and gives the magnitude and phase (starting point at time 0) of these sinusoidal signals. The DFT is often represented as a complex number with magnitude representing the magnitude of the sinusoid and the phase of complex variable as the phase of the sinusoid.

Why this decomposition?

Sinusoidal signals are periodic and hence easy to predict the future samples in time. Also a time series is represented using only few sinusoidal signals and thus we can easily obtain the future samples by extending these periodic signals.

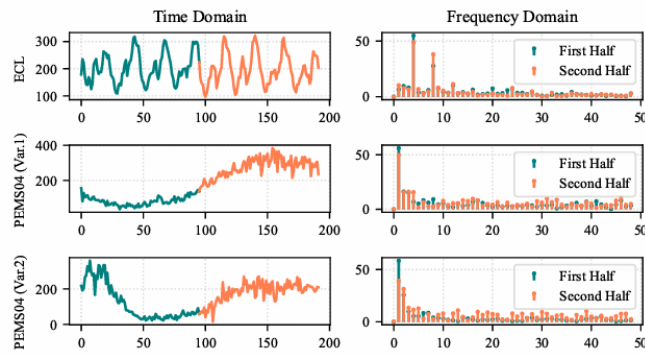


Figure 1: Time series and their corresponding frequency spectra. The series are normalized before applying the DFT, and the amplitudes of the frequency spectra are plotted. (1) The frequency spectra often exhibit strong consistency across adjacent temporal spans within the same time series, forming the basis for frequency-based forecasting. (2) Strong correlations between the two variables in PEMS04 (rows 2 and 3) are observed, suggesting that exploring such multivariate relationships could lead to more robust representations. (3) The frequency spectrum usually exhibits sparsity, with a few dominant frequencies.

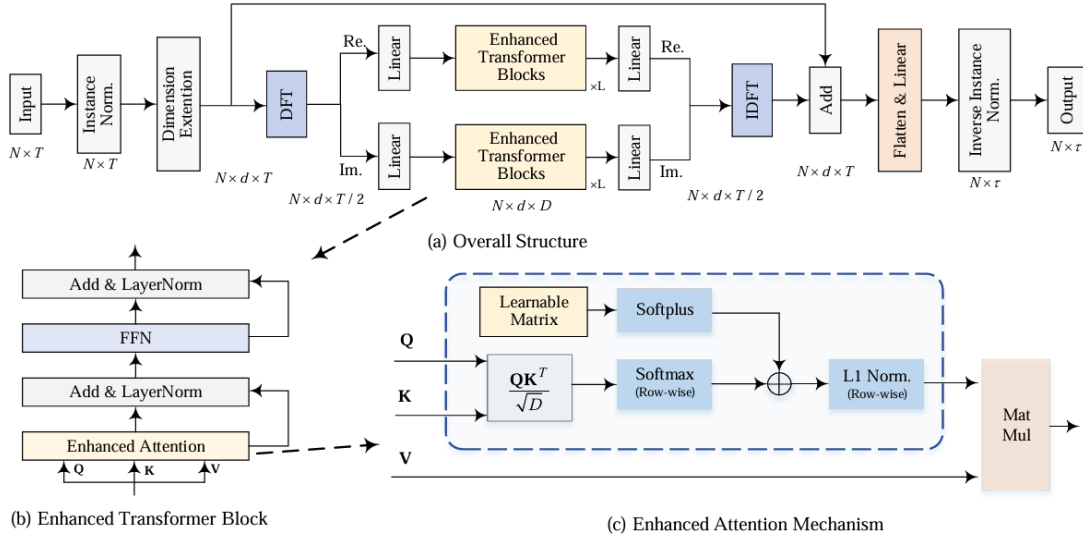


Figure 2: Overall structure of the FreEformer. We leverage the frequency spectrum to capture temporal patterns and employ an enhanced Transformer to model dependencies among multivariate spectra. The enhanced Transformer introduces a learnable matrix to the attention mechanism, which, as shown through theoretical analysis, addresses potential low-rank issues and improves gradient flow.

The **DFT** gives set of real and imaginary parts of the complex number representing the sinusoidal signals.

Enhanced Attention Block

Attention mechanism helps us to give importance to only those sinusoidal components that are important to predict the future samples. This is same attention concept of the transformers. However, a key limitation of vanilla attention in time-series (especially frequency domain) is the attention matrix becomes low-rank and overly sparse, which limits modeling capacity. To avoid this a learnable matrix B is added into the attention calculation to improve the learning capacity. The enhanced attention is calculated as

$$\text{Norm} \left(\text{Softmax} \left(\frac{QK^T}{\sqrt{D}} \right) + \text{Softplus}(B) \right) V,$$

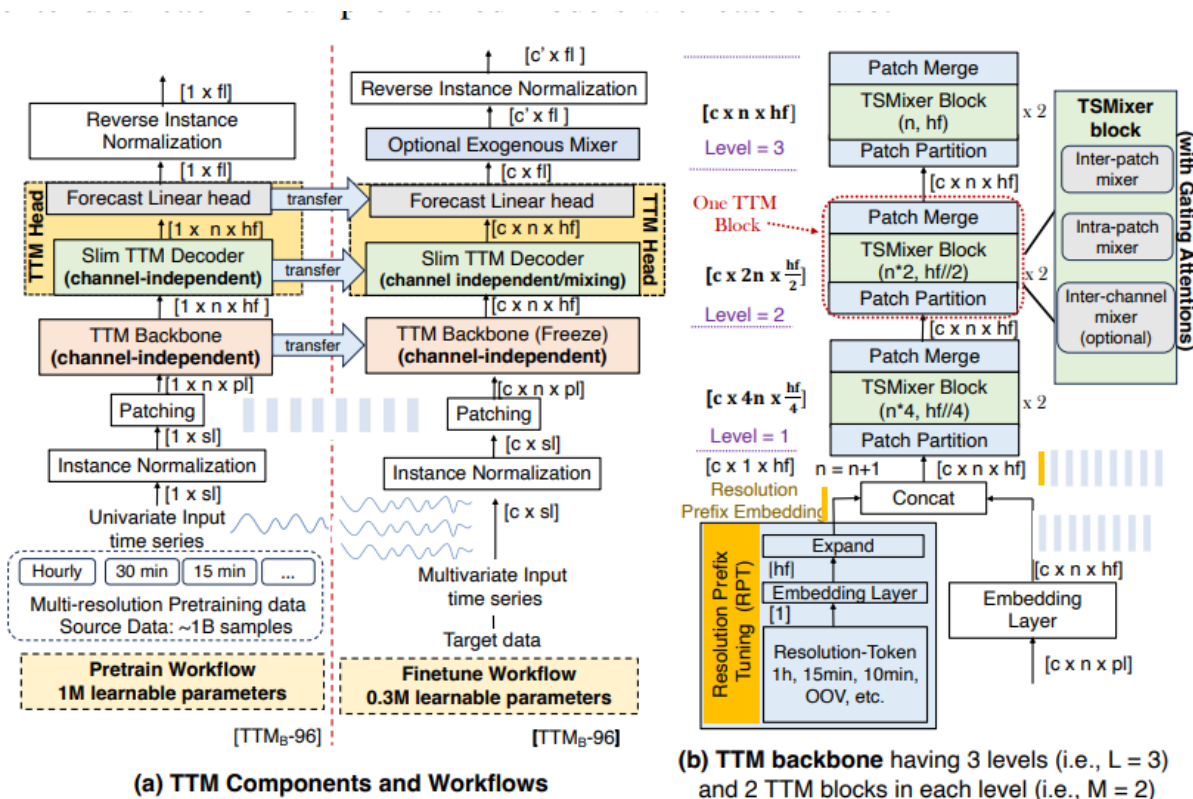
IDFT is done to get back to domain of the time series. There is a residual connection as in transformer model.

Model	FreEformer (Ours)		Leddam [2024]		CARD [2024c]		Fredformer [2024a]		iTrans. [2024a]		TimeMixer [2024b]		PatchTST [2023]		Crossfm. [2023]		TimesNet [2023a]		FreTS [2024c]		DLinear [2023]	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	0.379	0.381	0.386	0.397	0.383	0.384	0.384	0.395	0.407	0.410	0.381	0.395	0.387	0.400	0.513	0.496	0.400	0.406	0.407	0.415	0.403	0.407
ETTm2	0.272	0.313	0.281	0.325	0.272	0.317	0.279	0.324	0.288	0.332	0.275	0.323	0.281	0.326	0.757	0.610	0.291	0.333	0.335	0.379	0.350	0.401
ETTh1	0.433	0.431	0.431	0.429	0.442	0.429	0.435	0.426	0.454	0.447	0.447	0.440	0.469	0.454	0.529	0.522	0.458	0.450	0.488	0.474	0.456	0.452
ETTh2	0.372	0.393	0.373	0.399	0.368	0.390	0.365	0.393	0.383	0.407	0.364	0.395	0.384	0.405	0.942	0.684	0.414	0.427	0.550	0.515	0.559	0.515
ECL	0.162	0.251	0.169	0.263	0.168	0.258	0.176	0.269	0.178	0.270	0.182	0.272	0.208	0.295	0.244	0.334	0.192	0.295	0.202	0.290	0.212	0.300
Exchange	0.354	0.399	0.354	0.402	0.362	0.402	0.333	0.391	0.360	0.403	0.387	0.416	0.367	0.404	0.940	0.707	0.416	0.443	0.416	0.439	0.354	0.414
Traffic	0.435	0.251	0.467	0.294	0.453	0.282	0.433	0.291	0.428	0.282	0.484	0.297	0.531	0.343	0.550	0.304	0.620	0.336	0.538	0.328	0.625	0.383
Weather	0.239	0.260	0.242	0.272	0.239	0.265	0.246	0.272	0.258	0.279	0.240	0.271	0.259	0.281	0.259	0.315	0.259	0.287	0.255	0.298	0.265	0.317
Solar	0.217	0.219	0.230	0.264	0.237	0.237	0.226	0.262	0.233	0.262	0.216	0.280	0.270	0.307	0.641	0.639	0.301	0.319	0.226	0.254	0.330	0.401
PEMS03	0.102	0.206	0.107	0.210	0.174	0.275	0.135	0.243	0.113	0.221	0.167	0.267	0.180	0.291	0.169	0.281	0.147	0.248	0.169	0.278	0.278	0.375
PEMS04	0.094	0.196	0.103	0.210	0.206	0.299	0.162	0.261	0.111	0.221	0.185	0.287	0.195	0.307	0.209	0.314	0.129	0.241	0.188	0.294	0.295	0.388
PEMS07	0.080	0.167	0.084	0.180	0.149	0.247	0.121	0.222	0.101	0.204	0.181	0.271	0.211	0.303	0.235	0.315	0.124	0.225	0.185	0.282	0.329	0.395
PEMS08	0.110	0.194	0.122	0.211	0.201	0.280	0.161	0.250	0.150	0.226	0.226	0.299	0.280	0.321	0.268	0.307	0.193	0.271	0.212	0.297	0.379	0.416

It can be seen that it is performing better than all the previous models including DLinear, Path TST, and TimeMixer.

Tiny Time Mixers (2024, 196 citations)

It provides one foundational model for time series forecasting and helps in transfer learning (Train on large time series data (not the target data) and then fine tune on small amount of target data. It uses TSMixer and patching concepts to build the model and finetune on the target data to improve the performance.



The TTM model uses a multi-level structure with four main components. The backbone is the core part, built on TSMixer, which uses simple MLP blocks and lightweight attention to efficiently mix information across time, patches, and features, with added improvements to handle different data resolutions. The decoder has a similar design but is much smaller (about 10–20% of the backbone) and helps refine the learned representations. The forecast head is a simple linear layer that generates the final predictions. Additionally, an optional exogenous mixer can be used to include external data such as weather or events. The decoder and forecast head together form the TTM head, which is mainly updated during fine-tuning for specific tasks.

Figure a has the pretrain and finetune parts, and Figure b shows the details of the TTM backbone. The model uses a backbone mode that will be frozen after pretraining using a large dataset (1 B samples).

Adaptive Patching (AP): Adaptive patching means the model does not use a single fixed way of splitting the time series. Instead, it processes the data at multiple patch sizes across different layers, helping capture both short-term details and long-term patterns.

Components of Adaptive Patching

1. Patch Partition Block: This block reshapes the data into more patches with smaller size. It increases detail by breaking the signal into finer segments.

2. TSMixer Block: This block learns patterns by mixing information across time and variables. It processes the patches at the current level of detail.

3. Patch Merging Block: This block reconstructs the patches back to the original format. It prepares the data to move to the next level.

Different levels for different patch size

- Level 1 (coarse level):
Uses larger patches → captures overall trends and long-term patterns
- Level 2 (medium level):
Uses moderate patch sizes → captures mid-level patterns
- Level 3 (fine level):
Uses smaller patches → captures detailed and short-term variations

Diverse Resolution Sampling (DRS):

The data is converted into multiple time resolutions (e.g., seconds → minutes → hours).

This prevents the model from focusing only on high-resolution data and improves learning across scales.

Resolution Prefix Tuning (RPT):

The model is explicitly told the time resolution using a learned embedding.

This helps it quickly understand and adapt to different data resolutions, especially when data is limited.

Results

Data	Pre-trained models fine-tuned on 5% data					Other popular architectures trained on 5% data					
	TTM _B (Ours)	TTM _E (Ours)	TTM _A (Ours)	GPT4TS (NeurIPS '23)	Time-LLM (ICLR '24)	PatchTST (ICLR '23)	TSMixer (KDD '23)	TimeMixer (ICLR '24)	iTransformer (ICLR '24)	TimesNet (ICLR '23)	Dlinear (AAAI '23)
ETTH1	0.383	0.385	0.386	0.682	0.627	0.695	0.635	1.088	0.756	0.926	0.75
ETTH2	0.324	<u>0.318</u>	0.314	0.401	0.382	0.439	0.385	0.508	0.437	0.464	0.828
ETTM1	0.376	0.378	0.361	0.472	0.425	0.526	0.479	0.578	0.568	0.717	0.401
ETTM2	0.272	<u>0.268</u>	0.253	0.308	0.274	0.314	0.297	0.34	0.309	0.344	0.399
Weather	0.234	0.24	0.229	0.263	0.261	0.27	0.268	0.317	0.297	0.298	0.264
Electricity	0.183	0.207	0.18	0.178	<u>0.177</u>	0.176	0.197	0.239	0.202	0.402	<u>0.177</u>
Traffic	0.433	0.437	0.49	0.434	<u>0.423</u>	0.418	0.435	0.503	0.452	0.867	0.451
Size	1M	4M	5M	84M	7B						
TTM _B <i>f-imp</i> (%) <i>s-imp</i> (X)				15% ↑ 84X ↑	10% ↑ 7KX ↑	17% ↑	15% ↑	31% ↑	22% ↑	40% ↑	23% ↑
TTM _E <i>f-imp</i> (%) <i>s-imp</i> (X)				13% ↑ 21X ↑	8% ↑ 1.7KX ↑	15% ↑	13% ↑	30% ↑	20% ↑	40% ↑	21% ↑
TTM _A <i>f-imp</i> (%) <i>s-imp</i> (X)				15% ↑ 17X ↑	11% ↑ 1.4KX ↑	17% ↑	15% ↑	32% ↑	22% ↑	41% ↑	23% ↑

Table 4: **Few-shot 5%**. MSE averaged across $FL \in \{96, 192, 336, 720\}$, models are trained with 5% train data

We can see that TTM models perform better when the finetuned only on 5% of target data. Also observe that TTM size is 1M, 4M, and 5M parameters.